



Universitat
de les Illes Balears

TRABAJO DE FIN DE GRADO

SPORTS LEAGUE: DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA PARA LA GESTIÓN DE LIGAS DEPORTIVAS

Carlos Gálvez Mena

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Año académico 2025-26

SPORTS LEAGUE: DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA PARA LA GESTIÓN DE LIGAS DEPORTIVAS

Carlos Gálvez Mena

Trabajo de Fin de Grado

Escola Politècnica Superior

Universitat de les Illes Balears

Año académico 2025-26

Paraules clau del treball: Liga deportiva, Aplicación multiplataforma, Spring Boot, React Native, TFG

Tutor: Antonio Crespí Tobeña

Gracias a todos los profesores que me han ayudado a lo largo de mi carrera, y en especial a mi primer tutor Raimon Massanet Vila por haber confiado en mi idea y, mi segundo tutor, Antonio Crespí Tobeña por su apoyo y orientación durante la realización de este trabajo, aun habiendo tenido que coger el relevo cuando el primer tutor tuvo que ser relevado por motivos burocráticos.

ÍNDICE GENERAL

Índice general	iii
Acrònims	v
Resumen	vii
Resum	ix
1 Introducción	1
1.1 Alternativas actuales	2
1.1.1 Winner	2
1.1.2 Clupik	3
1.2 Motivación y objetivos	6
1.3 Estructura del documento	6
2 Gestión del proyecto	9
2.1 Metodología	9
3 Análisis y diseño del sistema	11
3.1 Especificación	11
3.1.1 Análisis de los interesados	11
3.1.2 Recogida de requisitos	12
3.1.3 Análisis	18
3.1.4 Reunión con la OUSIS	25
3.1.5 Requisitos funcionales	25
3.2 Diseño	36
3.2.1 Arquitectura del sistema	36
3.2.2 Diseño del sistema	36
3.2.3 Diseño de la base de datos	36
3.2.4 Diseño de la interfaz	38
3.2.5 Diseño del contrato API	39
3.2.6 Control de versiones	39
4 Estado del arte en el ámbito tecnológico	41
4.1 Servidor back-end	41
4.2 Base de datos	42
4.3 Almacenamiento de objetos (Object Storage)	44
4.4 Aplicación front-end	45

5	Implementación	47
5.1	Entorno de desarrollo	47
5.1.1	Editor de código	47
5.1.2	Ejecución y despliegue	47
5.1.3	Migración	47
5.2	Servidor back-end	48
5.3	Base de datos	48
5.4	Almacenamiento de objetos	48
5.5	Interfaz multiplataforma	48
6	Conclusiones	49
6.1	Aprendizaje	49
6.2	Mejoras aportadas con respecto a las alternativas	49
6.3	Trabajo futuro	50
A	Formularios de opinión	51
B	Resultados de los formularios	53
C	Interfaz Winner	57
D	Interfaz Clupik	61
E	Diseño de la base de datos	67
F	Contrato API	69
	Bibliografía	71

ACRÒNIMS

TFG Treball Final de Grau

GEIN Grau d'Enginyeria Informàtica

UIB Universitat de les Illes Balears

OUSIS Oficina d'Universitat Saludable i Sostenible

LVUIB Liga de Voleibol de la Universitat de les Illes Balears

SaaS Software as a Service, Software como Servicio

OS Object Storage, Almacenamiento de Objetos

BD Base de Datos

UX User Experience, Experiencia de Usuario

DTO Data Transfer Object, Objeto de Transferencia de Datos

ORM Object-Relational Mapping, Mapeo Relacional de Objetos

ABAC Attribute-Based Access Control, Control de Acceso Basado en Atributos

XSS Cross-Site Scripting

FVIB Federació de Voleibol de les Illes Balears

FIVB Fédération Internationale de Volleyball

RFEVB Real Federación Española de Voleibol

ACID Atomicidad, Consistencia, Aislamiento y Durabilidad

ENS Esquema Nacional de Seguridad

OAuth Open Authorization

RESUMEN

[spa] Este trabajo consiste en la comparación de las aplicaciones utilizadas por la UIB para la gestión de la liga de voleibol con respecto a nuevas posibles soluciones. Se basa en el análisis de los pros y contras de cada aplicación utilizada y ofrece una solución nueva y mejorada cuyas funcionalidades se basan en las necesidades y opiniones recogidas de los principales usuarios. También explica la gestión de un proyecto de software de forma completa: desde la recogida de requisitos a partir de encuestas de opinión, pasando por el diseño de la arquitectura e interfaz y acabando con la implementación y evaluación del comportamiento de la nueva solución.

RESUM

[cat] Aquest treball consisteix en la comparació de les aplicacions utilitzades per la UIB per a la gestió de la lliga de voleibol respecte a noves possibles solucions. Es basa en l'anàlisi dels pros i contres de cada aplicació utilitzada i ofereix una solució nova i millorada les funcionalitats de la qual es basen en les necessitats i opinions recollides dels principals usuaris. També explica la gestió d'un projecte de programari de manera completa: des de la recollida de requisits a partir d'enquestes d'opinió, passant pel disseny de l'arquitectura i la interfície i acabant amb la implementació i l'avaluació del comportament de la nova solució.

INTRODUCCIÓN

La gestión de ligas deportivas universitarias puede resultar complicado si no se anticipan varios puntos como lo pueden ser: qué deportes ofrece la universidad interesada para poder celebrar ligas, qué formato seguirán dichas ligas (todos contra todos con grupos, fase eliminatoria, etc.), qué tipo de personas pueden participar (¿cualquier persona o solo gente relacionada con la universidad?), qué restricciones deben tener los equipos y jugadores que participan dentro de la liga, qué sistema de puntuación se debe utilizar, y muchos otros aspectos. Una planificación incorrecta en cualquiera de los puntos mencionados puede acarrear problemas de organización que lleven a un desarrollo de la liga que no acabe siendo satisfactorio para ningún interesado ya sea para los integrantes universitarios que participen o la misma universidad gestora.

Esto mismo es lo que ha ocurrido con la Liga de Voleibol de la Universitat de les Illes Balears (LVUIB). A lo largo de los años que se ha celebrado, los participantes y gestores se han dado cuenta de lo costoso que es encontrar un sistema a nivel de aplicación que permita organizar y gestionar una liga deportiva. Suelen ser necesarias muchas funcionalidades avanzadas y personalizadas que llevan a optar por soluciones que acaban necesitando una inversión monetaria, pero que aun así no acaban de convencer a los participantes y usuarios de dicho sistema.

Debido a esta situación se llegó a la conclusión de que era posible y necesario desarrollar un sistema que permitiera una gestión centralizada evitando los problemas comentados y que, además, permitiera utilizar la aplicación como el único punto necesario de gestión.

Cabe remarcar que la Oficina d'Universitat Saludable i Sostenible (OUSIS) es el organismo encargado de fomentar y gestionar los eventos deportivos y relacionados con la salud que se celebran dentro del entorno de la Universitat de les Illes Balears (UIB), como pueden ser las ligas universitarias de fútbol, voleibol y pádel. Esta gestión no estuvo tan presente durante el primer año que se celebró la LVUIB, ya que fue un evento fomentado y gestionado por una asociación de alumnos creada al uso para dicho evento. En años posteriores, a medida que la LVUIB fue creciendo en popularidad y participantes, la OUSIS decidió involucrarse de lleno.

1.1 Alternativas actuales

Aunque actualmente podemos encontrar miles de alternativas que aportan las funcionalidades básicas y genéricas necesarias para una correcta gestión de ligas deportivas, nos centraremos en las aplicaciones utilizadas en las temporadas anteriores de la LVUIB. Dichas aplicaciones son: Winner y Clupik [1, 2].

Las explicaciones de las funcionalidades de cada aplicación se basan en las imágenes que se pueden encontrar en los anexos correspondientes: C y D

1.1.1 Winner

Winner se utilizó en los primeros dos años de celebración de la LVUIB. Es una aplicación que se puede encontrar en cualquier tienda de aplicaciones móviles. De base, su uso es gratuito ofreciendo funcionalidades básicas, pero suficientes para organizar ligas deportivas amateurs. Las funcionalidades que nos ofrece son las siguientes:

Registro e inicio de sesión

El registro de sesión es sencillo. Permite crear una cuenta e iniciar sesión de forma básica utilizando un email y contraseña. También es posible iniciar sesión a través de una cuenta de Google.

Creación y gestión de ligas

Permite la creación de ligas ofreciendo una gran variedad de deportes y distintos tipos de participantes(individual, equipos, dobles, etc.). Es posible crear ligas dadas plantillas predeterminadas o como consecución de fases de distintos formatos: árbol de eliminatoria y liga con o sin grupos. También es posible personalizar el sistema de puntuación y generar los partidos a partir de un calendario en el que se deciden las veces que se enfrentan cada equipo.

Contras: es una aplicación de gestión completamente local, es decir, la gestión recae sobre una sola persona que es la que posee el dispositivo donde se ha creado dicha liga. Para que otras personas puedan hacer un seguimiento de la liga es necesario compartir con ellos, de forma directa, el enlace que se genera al crear la liga y, a su vez, los interesados en visualizar la liga necesitan iniciar sesión. El sistema de puntuación es útil, pero básico; por ejemplo, en voleibol, no se podría puntuar de forma similar a como se hace de forma oficial.

Gestión de equipos y jugadores

Ofrece la posibilidad de registrar equipos y jugadores con sus respectivos nombres. También es posible asignarle logotipos a cada equipo y jugador a partir de imágenes básicas con colores genéricos y banderas de países que ofrece la aplicación

Contras: el registro de los equipos y jugadores es local y genérico, lo que puede ocasionar confusión si se están gestionando varias ligas al mismo tiempo. La personalización es limitada dado que no es posible asignar imágenes propias como escudos o fotos de perfil.

Visualización de Resultados y Clasificación

Es posible registrar los resultados de los partidos de forma manual: ubicación, ganador y el minutaje de anotación de cada punto. Lo cual hace que se actualice la tabla de clasificación del grupo pertinente. En cada tabla clasificatoria es posible visualizar las estadísticas generales de los equipos, además, Winner permite ver la evolución de las estadísticas de los equipos durante el desarrollo de la liga mediante diferentes gráficos.

Contras: ninguno, es posible aprender de los puntos fuertes de esta sección para poder aplicarlos a un nuevo sistema.

Opcionalidad de pago

Adicionalmente, contamos con la opción de invertir una pequeña cantidad monetaria en caso de querer ampliar las funcionalidades. Las ventajas que proporciona la aplicación con su versión de pago y las ofertas existentes las podemos ver en la figura C.5.

Contras: la versión de pago se basa en un servicio de suscripción que, aunque sea asequible, hace necesario un pago recurrente para mantener las funcionalidades extra.

En resumen, Winner, es una aplicación de gestión que cubre con creces las necesidades de ligas de nivel amateur, pero no es suficiente para ligas deportivas que quieran profesionalizarse. La responsabilidad recae únicamente en la persona que posee la aplicación y dispositivo sobre el que se va a efectuar la creación, registro y gestión de la liga, no es posible compartir la carga de trabajo entre varios administradores. Esto hace que no sea posible escalar su uso en la LVUIB debido a la necesidad de cumplir con los estándares derivados de ligas profesionales, como las gestionadas por la Federació de Voleibol de les Illes Balears (FVIB)

1.1.2 Clupik

Clupik se presenta como un Software as a Service, Software como Servicio (SaaS) que ofrece la posibilidad de gestionar tanto clubes como ligas deportivas, pero en este caso, nos centraremos en la parte de más nos concierne, las ligas. La principal función de esta herramienta es la de permitir a cualquier persona crear y administrar una liga deportiva. Para poder hacerlo, pone a disposición de los usuarios las siguientes funcionalidades:

Registro e inicio de sesión

Es muy similar, por no decir casi idéntico, al sistema de Winner: creación de cuenta e inicio de sesión a partir de un email y una contraseña. Aunque en este caso no tenemos la posibilidad de enlazar una cuenta de Google. La parte positiva es que el inicio de sesión no es un requisito para poder consultar ligas públicas.

Creación y gestión de ligas

Se puede ver que la creación de una liga es similar a Winner aunque con algunas diferencias. Al crear la liga puedes elegir entre deportes y juegos de competición, cosa que Winner no contempla; aunque Clupik no permite elegir el formato de participación, solo se puede jugar dentro de un equipo. Clupik, al igual que Winner, permite la creación

de ligas como sucesiones de fases, pero Clupik no tiene plantillas predefinidas como lo hace Winner. Es posible definir los partidos en los dos tipos de fases como podemos observar en las figuras D.2d y D.2e. Por último, es posible editar manualmente los equipos que se enfrentan en cualquier partido generado.

Contras: la mayor contraprestación que se puede apreciar es que para gestionar una liga, desde el papel de administrador, es necesaria una aplicación adicional, que se sincroniza con la aplicación principal, a la que existe para poder visualizar la liga desde el papel de espectador o participante. Y también existe otra aplicación para los árbitros que se centra únicamente en la gestión de los partidos. En resumen: existen 2 aplicaciones diferentes para asegurar una correcta gestión de la liga.

Gestión de equipos y jugadores

La creación tanto de equipos como jugadores es muy básica: nombre en caso de los equipos y nombre y apellido en caso de jugadores.

Contras: no es posible personalizar una foto de perfil de jugador o escudo del equipo con las funcionalidades gratuitas de Clupik.

Visualización de ligas

Para poder ver una liga, primero es necesario buscarla por su nombre o el nombre del organizador. Una vez que seleccionas una liga puedes consultar: los próximos partidos y los resultados de los partidos ya jugados, los diferentes grupos que hay, los equipos existentes, los jugadores por equipo, la tabla clasificatoria para cada grupo y las estadísticas de la liga. También es posible visualizar las jornadas que constituyen la liga y qué partidos se celebran en cada jornada.

Cabe destacar que existe una aplicación adicional para poder visualizar el desarrollo de la liga, aunque es su única funcionalidad. La visualización incluye todas las ligas activas de la UIB, aunque es posible aplicar una serie de filtros para centrarnos únicamente en el contenido que nos interesa (Ver D.6). También tiene la opción de iniciar sesión con unas credenciales válidas de la UIB para ver el contenido relacionado con la participación como usuario, por ejemplo: el equipo donde juega el usuario o los partidos en los que participa el equipo del usuario.

Esto hace que existan un total de dos aplicaciones que sirven para consultar los datos de la liga. Pero la segunda solo muestra datos relacionados con los próximos partidos y la tabla de clasificación. Es decir, la segunda aplicación es una versión reducida de la primera añadiendo un sistema completo de filtros además de mostrar otro contenido referente a la OUSIS y presentar un diseño similar al de aplicaciones de la UIB

Contras: el principal punto negativo de esta sección es la existencia de dos aplicaciones diferentes. Esto puede hacerle creer al usuario que la segunda aplicación es una versión incompleta de la primera o que incluso crea que la primera es algo completamente agnóstico a la UIB al carecer el diseño propio de esta. Además, acarrea una carga cognitiva a los usuarios porque, al existir dos opciones no es intuitivo saber cuál de las dos ofrece toda la información de forma correcta.

Opcionalidad de pago

Se puede ver como Clupik ofrece dos tipos de planes de pago para extender las funcionalidades del sistema en caso de que las funcionalidades que vienen por defecto no sean suficientes para cumplir con las necesidades de gestión. Observamos como la oferta del plan *TOP* ofrece un conjunto de funcionalidades que permiten una completa gestión de ligas a todos los niveles: árbitros, administradores, cuerpo técnico y demás opciones que ayudan a hacer una gestión profesional.

Contras: el único contra y el más claro de esta sección es el elevado coste que ha supuesto el uso de este sistema durante la última temporada.

Uso en la LVUIB

Winner fue la aplicación que se utilizó durante los dos primeros años de celebración de la LVUIB. El primer año que se celebró los organizadores decidieron utilizar las funcionalidades básicas, mientras que el segundo año se optó por la versión de pago debido a la necesidad de mayor personalización, mejora de la organización y eliminación de los frecuentes anuncios que resultaban una molestia para el principal administrador. La opción de compra que se adquirió fue una que ya no está disponible: una compra única con un precio no mayor a 15 €.

Clupik se ha utilizado durante el tercer año (coincidiendo con el año académico 2025-2026) y ha representado un cambio importante respecto a la primera alternativa. Se presenta como un ‘cambio importante’ porque ha sido la propia OUSIS la que ha decidido implicarse más en la organización y ha sido la principal gestora de la LVUIB esta última temporada.

La OUSIS optó por el plan de pago más completo y funcional: el plan *TOP*. Cabe destacar que el plan obliga a hacer el pago de forma anual. Esto hace que, como la LVUIB se celebra durante los dos semestres de un año académico, el coste total del sistema ha sido de 599.04 €.

Para esta aplicación no se ha hecho mucho hincapié en los aspectos negativos, ya que todos ellos han sido remarcados y explicados por los propios usuarios en la sección 3.1.2.

Para acabar de entender esta sección podemos apreciar en la siguiente tabla una comparación entre las aplicaciones y sus funcionalidades y cuáles de ellas se pretende mejorar con el nuevo sistema:

Aplicaciones Funcionalidades	Winner	Clupik
Registro e inicio de sesión	No	No
Creación y gestión de ligas	Parcial	Parcial
Gestión de equipos y jugadores	Sí	Sí
Visualización	Parcial	Sí
Opcionalidad de pago	Sí	Sí

1.2 Motivación y objetivos

La motivación principal de este Treball Final de Grau (TFG) es la de diseñar e implementar un sistema gestor de ligas deportivas que aporte una mejora con respecto a las alternativas que se han estado utilizando hasta ahora dentro de la LVUIB. Esta mejora se basará en las opiniones de los usuarios que utilizaron una de las alternativas.

Los objetivos principales que han buscado cumplir a través del desarrollo de Sports League es:

Objetivo General

Desarrollar un sistema de gestión integral cliente-servidor para digitalizar y automatizar la administración de la liga deportiva universitaria, mejorando la comunicación y los flujos de trabajo entre todos los interesados. Y, además, asegurar que el sistema no requiere de herramientas externas ni diversas aplicaciones para poder funcionar.

Objetivos Específicos

OE1 Recopilar las necesidades de los distintos perfiles dentro de la LVUIB.

OE2 Diseñar un sistema que permita cumplir con las necesidades recopiladas.

OE3 Implementar y desarrollar este sistema enfocado al uso multiplataforma.

OE4 Crear una estrategia de despliegue escalable y robusta.

El cumplimiento de los objetivos específicos contribuye al correcto cumplimiento del objetivo general, ya que proporcionan la secuencia de hitos y resultados intermedios que se deben alcanzar. Cada objetivo específico se corresponde con la salida esperada de los grupos de procesos, cuya consecución acumulativa garantiza el cumplimiento del objetivo general.

1.3 Estructura del documento

Este documento sirve para entender el proceso de trabajo aplicado para el desarrollo de Sports League. Por ello cada capítulo explica una parte en concreto del proyecto.

El primer capítulo explica el contexto y las motivaciones del proyecto: la necesidad que tiene la OUSIS de gestionar la LVUIB, qué soluciona y qué aporta con respecto a las soluciones existentes que se utilizaron y que no llegaron a convencer a los interesados.

El segundo capítulo detalla la planificación del proyecto seguida para completarlo, que incluye: interesados, el ciclo de vida, entregables, estimación de tiempo, gestión de calidad y gestión de cambios.

El tercer capítulo describe las especificaciones del sistema tanto a nivel funcional como arquitectónico. Explica el proceso de recogida de requisitos a partir del uso de formularios de satisfacción y el diseño del sistema necesario para cumplir con dichos requisitos.

El cuarto capítulo muestra cómo ha sido la implementación del sistema: si se ha cumplido con lo establecido en la planificación, además de la arquitectura de software, patrones de diseño y principios SOLID [3] que se han aplicado.

El quinto y último capítulo cuenta las conclusiones resultado de todo el trabajo completado: qué aprendizaje ha habido, qué se ha aportado con respecto a las tecnologías existentes y qué trabajo se realizará en el futuro utilizando este proyecto como base.

GESTIÓN DEL PROYECTO

2.1 Metodología

Para completar el proyecto se ha optado por un desarrollo en cascada. Este enfoque nos permite asegurar que un paso está completo antes de pasar al siguiente. No se ha optado por metodologías ágiles, ya que incurren en procesos no tan necesarios cuando no tenemos un equipo de trabajo y la organización del proyecto recae en una sola persona.

La planificación del proyecto puede apreciarse mediante el siguiente cronograma:

ANÁLISIS Y DISEÑO DEL SISTEMA

En este capítulo veremos todos los pasos que se han tomado para completar la especificación y diseño del sistema. Abarca desde el análisis de stakeholders y recogida de requisitos a partir de formularios de opinión hasta el diseño completo del sistema.

3.1 Especificación

3.1.1 Análisis de los interesados

Entre los interesados del proyecto: todas aquellas partes que se ven, de una forma u otra, afectadas por el ciclo de vida del proyecto, podemos apreciar dos grupos de interesados claramente divididos. Por un lado, tenemos los interesados institucionales y académicos, que velan por el correcto desarrollo del proyecto; mientras que, por otro lado, están los usuarios finales, aquellos que interactuarán directamente con el sistema a través de los distintos roles definidos. Todos los interesados quedan explicados en la tabla 3.1.

Cuadro 3.1: Stakeholders del sistema

Stakeholder	Usuario	Descripción del Rol	Intereses y Objetivos
OUSIS	No	Oficina de la UIB beneficiada por el desarrollo del proyecto.	Obtener una herramienta funcional y de bajo coste que digitalice la gestión deportiva de la universidad, asegurando viabilidad y optimización.

Continúa en la página siguiente

Cuadro 3.1 – continuación de la página anterior

Stakeholder	Usuario	Descripción del Rol	Intereses y Objetivos
Tutor / Tribunal	No	Evaluación y seguimiento académico del proyecto.	Garantizar el cumplimiento de los requisitos metodológicos, arquitectónicos y de ingeniería de software para la obtención del título.
Usuario / Espectador	Sí	Usuario común del sistema con o sin cuenta registrada.	Consultar y seguir la información pública de las competiciones y ligas activas.
Jugador	Sí	Participante inscrito en una o varias ligas.	Consultar calendarios, normativas y revisar clasificaciones o resultados de sus equipos.
Capitán	Sí	Representante del equipo ante la organización y otros clubes.	Inscribir jugadores y gestionar acuerdos o cambios de partidos de forma centralizada.
Árbitro	Sí	Responsable de la validez de encuentros y reporte de actas.	Disponer de una interfaz rápida para cumplimentar actas digitales y registrar resultados.
Administrador	Sí	Gestor principal de ligas, calendarios e incidencias.	Automatizar calendarios, resolver incidencias y reducir la carga de gestión manual.

3.1.2 Recogida de requisitos

Los requisitos funcionales han sido generados basándonos en las opiniones que tenían los participantes de la LVUIB acerca del sistema que se ha estado utilizando: Clupik. Las respuestas hacen referencia tanto a las aplicaciones de Clupik como a la aplicación adicional de visualización de la UIB. Para completar la recogida de dichos requisitos se optó por la distribución de formularios de opinión y sugerencias. Las preguntas formuladas (que se pueden encontrar en los anexos) se agrupan en 4 formularios distintos según el rol que tuviera el participante dentro de la liga: jugador, capitán, árbitro y administrador.

Todos los formularios tienen en común estas cuestiones:

Aspectos cuantitativos

¿Cada cuánto usas la aplicación de la liga de la UIB o consultas la información de la liga a través de ella?

Pregunta de elección única cuyas opciones son:

- Más de 4 días a la semana.
- Menos de 4 días a la semana.
- Un día a la semana.
- Menos de 2 veces al mes.
- No sabía que había una aplicación o me informo a través de otras personas.

Con esta pregunta es posible hacerse una idea sobre lo importante que es el uso de la aplicación. Un uso bajo indica que la aplicación tiene una utilidad mínima o que incluso no la tiene, mientras que un uso alto indica que la aplicación es indispensable para la gestión de la liga.

¿A través de qué medio accedes principalmente a la aplicación?

Pregunta de elección única cuyas opciones son:

- Navegador web.
- Aplicación móvil.
- No accedo.

Nos aporta información con respecto al formato que usan los usuarios para acceder al sistema. Sirve para saber sobre qué dispositivos y plataformas enfocar el desarrollo del sistema.

Aspectos cualitativos

Usabilidad *Define del 1 al 10 lo cómodo que es utilizar la aplicación y navegar por ella.*

Pregunta con escala de Likert (1-10) que va de *Pésima* a *Excelente*.

Accesibilidad *Define del 1 al 10 cuan fácil ha sido aprender a usarla.*

Pregunta con escala de Likert (1-10) que va de *He aprendido rápido* a *Todavía me cuesta saber cómo funciona*.

Manejo *Define del 1 al 5 cómo de rápido accedes al contenido/información que deseas ver.*

Pregunta con escala de Likert (1-5) que va de *Necesito menos de 3 clics* a *Me lío y tardo mucho*.

Diseño *¿Te gusta el diseño de la aplicación?*

Pregunta con escala de Likert (1-5) que va de *No es agradable de ver* a *Tiene un diseño atractivo*.

Estas cuatro preguntas aportan información en cuanto a la opinión que tiene los usuarios respecto a distintos aspectos funcionales de la aplicación.

Nota final y necesidad de un nuevo sistema

Nota final *¿Cómo puntuarías a nivel general la aplicación?*

Pregunta con escala de Likert (1-10) que va de *No me gusta mucho* a *No cambiaría nada*.

Tal y como indica la pregunta, muestra de forma directa la opinión que tienen los usuarios a nivel general.

¿Te gustaría una nueva aplicación que incluya nuevas y mejores funcionalidades?

Pregunta de elección única cuyas opciones son:

- Sí.
- No.

Esta sección permite saber la opinión general de los usuarios de la aplicación y si realmente necesitan un nuevo sistema que sustituya al actual.

Aspectos específicos

Cada formulario contiene una pequeña sección de cuestiones concretas relacionadas con el rol al cual se enfoca el formulario. Hay distintas preguntas según el rol.

Gestión equipos (Capitán) *¿Cómo puntuarías la gestión de tu equipo a través de la aplicación?*

Pregunta con escala de Likert (1-5) que va de *No la uso para nada* a *Excelente*.

Esta pregunta nos da información sobre el uso que hacen los capitanes de la aplicación.

¿Cómo puntuarías la gestión de los partidos a través de la aplicación? (Árbitro)

Pregunta con escala de Likert (1-5) que va de *Pésima* a *Excelente*.

Esta pregunta nos aporta información respecto a la facilidad que tienen los árbitros para arbitrar los partidos mediante la aplicación.

¿Usas algún dispositivo para rellenar el acta de los partidos? (Árbitro)

Pregunta de elección única cuyas opciones son:

- Tablet
- Móvil
- Solo uso acta en papel.

Con esta pregunta podemos saber si los árbitros utilizan algún dispositivo para rellenar el acta de los partidos o simplemente la rellenan a papel. Así podemos ver si es necesario incluir funcionalidades para la generación de actas digitales o incluso adaptar el diseño de la interfaz para otros tipos de dispositivos.

¿Cómo puntuarías la administración de la liga a través de la aplicación? (Administrador)

Pregunta con escala de Likert (1-5) que va de *Pésima* a *Excelente*.

Esta pregunta nos aporta información respecto a la facilidad que tiene los administradores para gestionar la liga a través de la aplicación.

¿Qué aspectos te gustaría que hubiera? Preguntas de elección múltiple cuyas opciones son:

Para jugadores:

- Aplicación móvil.
- Página web.
- Gestión del equipo en la propia aplicación.
- Solicitud para unirse a equipos dentro de una liga.
- Ninguna de las anteriores.

Para capitanes:

- Aplicación móvil.
- Página web.
- Gestión del equipo en la propia aplicación.
- Enviar solicitud para crear un equipo dentro de una liga.
- Enviar solicitud a jugadores para que se unan a mi equipo.
- Acuerdo de fecha del partido mediante propuestas.
- Crear incidencias: comportamiento árbitros, no imparcialidad, errores en el resultado, etc.
- Ninguna de las anteriores.

Para árbitros:

- Aplicación móvil.
- Página web.
- Crear incidencias: comportamiento de los jugadores de un equipo o del equipo en general.
- Registrar todos los datos del partido: puntos, sets, cambios, tarjetas, incidencias, etc. a través de la aplicación.
- Generación de acta electrónica al acabar un partido.
- Registrar mi disponibilidad para arbitrar.
- Enviar solicitud para arbitrar un partido.
- Rechazar el arbitraje concedido de un partido.
- Guardar mi firma para poder firmar los partidos de una manera más fácil y rápida.
- Ninguna de las anteriores.

Para administradores

- Aplicación móvil.
- Página web.
- Creación de ligas personalizadas: separación por distintas fases. Por ejemplo: liga 1, liga 2 (oro y plata), final four.
- Crear ligas a partir de formatos preestablecidos: Formato Real Federación Española de Voleibol (RFEVB), Formato Fédération Internationale de Volleyball (FIVB), etc.
- Personalización completa sobre las ligas: número de sets por partido, puntos por set, tiempos muertos.
- Resolución de incidencias creadas sobre jugadores o equipos con su correspondiente penalización.
- Asignar un árbitro a un partido de forma directa.
- Asignaciones automáticas de árbitros a partidos.
- Aceptar solicitudes de creación de equipos por parte de los jugadores.
- Ninguna de las anteriores.

Estas preguntas nos permiten saber qué funcionalidades les gustaría que hubiera a los usuarios según su rol. De esta forma es posible enfocar el diseño del sistema para cumplir con dicha separación de roles.

Preguntas abiertas

Otros aspectos *Pon cada funcionalidad que te gustaría que hubiera y que no haya aparecido en la lista anterior separada por comas. Ej.: Quiero [insertar funcionalidad] para poder/porque [insertar razón/justificación]*

Problemas encontrados *¿Qué aspectos has encontrado engorrosos o que no te parezcan bien? Por ejemplo: "Me cuesta llegar a la sección de clasificación de los equipos", etc.*

Preguntas de respuesta abierta que permiten a los encuestados detallar problemas específicos dentro de la aplicación además de especificar otras funcionalidades deseadas que no se contemplan en otras preguntas

En la siguiente tabla se muestra un resumen con las preguntas formuladas en los formularios:

Cuadro 3.2: Preguntas del cuestionario del sistema

Pregunta	Tipo	Rol
¿Cada cuánto usas la aplicación de la liga de la UIB o consultas la información de la liga a través de ella?	Elección única	Todos
¿A través de qué medio accedes principalmente a la aplicación?	Elección única	Todos
Usabilidad	Escala de Likert (1-10)	Todos
Accesibilidad	Escala de Likert (1-10)	Todos
Manejo	Escala de Likert (1-5)	Todos
Diseño	Escala de Likert (1-5)	Todos
Nota final	Escala de Likert (1-10)	Todos
¿Te gustaría una nueva aplicación que incluya nuevas y mejores funcionalidades?	Elección única	Todos
Gestión equipos	Escala de Likert (1-5)	Capitán
¿Cómo puntuarías la gestión de los partidos a través de la aplicación?	Escala de Likert (1-5)	Árbitro
¿Usas algún dispositivo para rellenar el acta de los partidos?	Elección única	Árbitro
¿Cómo puntuarías la administración de la liga a través de la aplicación?	Escala de Likert (1-5)	Administrador
¿Qué aspectos te gustaría que hubiera?	Elección múltiple	Todos
Otros aspectos	Respuesta abierta	Todos
Problemas encontrados	Respuesta abierta	Todos

Se puede apreciar que, para las preguntas de escala de Likert, la mayoría de las veces se usan escalas del al 10 a excepción de dos casos, que se usan escalas del 1 al 5: el diseño y el manejo de la aplicación. Esta diferencia está directamente relacionada con la naturaleza del aspecto evaluado. La mayoría de aspectos de escalas funcionan como evaluaciones estándar para facilitar a los encuestados la evaluación; pero, en los aspectos mencionados la escala funciona como una asociación directa con la pregunta. Para el manejo vemos como la escala representa la cantidad de clics necesarios para acceder a la información deseada donde una nota más baja indica una cantidad inferior de clics. Mientras que para el diseño se hace de este modo por comodidad del encuestado. En aspectos subjetivos como este, al encuestado le es más cómodo evaluar

del 1 al 5 que del 1 al 10, ya que de esa forma acarrea una menor carga cognitiva al necesitar menos argumentos internos para evaluar el aspecto.

Los formularios fueron distribuidos a través de WhatsApp a los distintos grupos objetivos y se esperó a que hubiera una cantidad significativa de respuestas.

3.1.3 Análisis

Las encuestas fueron respondidas por un total de 26 participantes: 16 jugadores, 2 árbitros, 6 capitanes y 2 administradores. Las respuestas presentadas en los formularios concluyeron en los siguientes resultados:

Aspectos cuantitativos

La gráfica B.1 nos permite ver que la mayoría de los usuarios casi no usan la aplicación o que incluso no la usan para nada. Esto puede ser debido a la poca utilidad o facilidad de uso de la aplicación que encuentran en su uso cotidiano.

Esto nos puede indicar que la aplicación tan solo la usan los usuarios que estrictamente tienen que utilizarla: como los árbitros, para guardar los resultados de los partidos; o los administradores, para hacer un seguimiento continuo del correcto desarrollo de la liga.

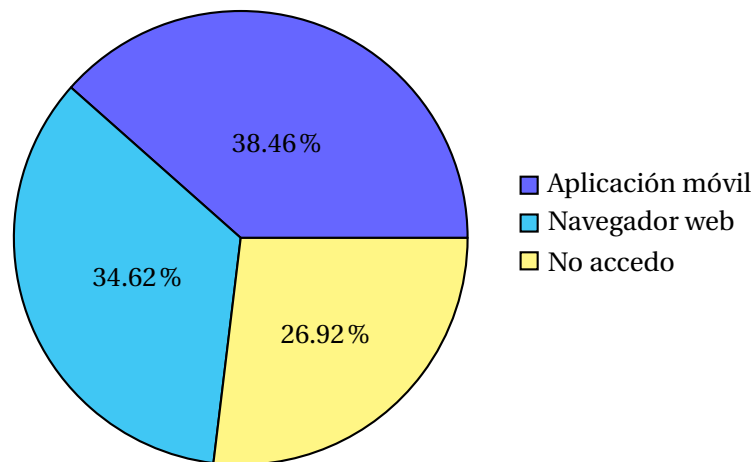


Figura 3.1: ¿A través de qué medio accedes principalmente a la aplicación?

La gráfica 3.1 nos confirma el uso multiplataforma que hacen los usuarios de la aplicación, ya sea para ver ciertos datos de forma rápida con dispositivos móviles o consultas más precisas y extensas con el navegador web. Este dato nos informa sobre la necesidad de un sistema disponible en dispositivos varios.

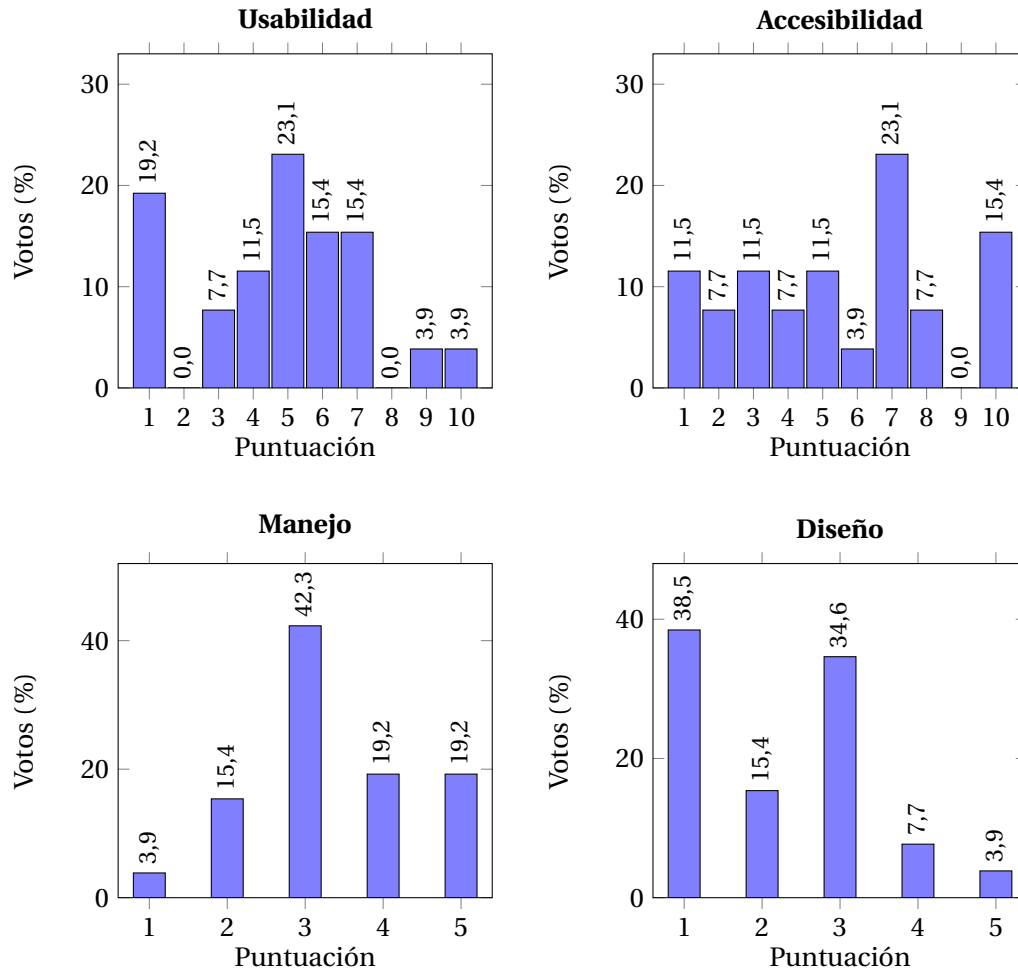
Aspectos cualitativos

Figura 3.2: Aspectos comunes

Las cuatro gráficas de 3.2 nos detallan cómo los usuarios sienten la interactividad con la aplicación. Esto nos puede ayudar a saber qué debemos cambiar respecto a la aplicación actual y qué podemos replicar y mejorar.

Podemos apreciar que a la mayoría de los usuarios no les desagrada la usabilidad. Con esto podemos intentar seguir un esquema de uso similar con el que cuenta la aplicación.

La accesibilidad parece seguir un patrón uniforme que crece conforme nos acercamos a valores que reflejan una menor accesibilidad. Lo que nos dice que la accesibilidad es un punto a mejorar necesitando crear un sistema intuitivo que no le genere fricción al usuario.

El manejo es puntuado mayoritariamente con un valor medio, pero que decrece conforme nos acercamos a las mejores puntuaciones. Esto nos dice que necesitamos un sistema que ofrezca al usuario una distribución de los datos sencilla y entendible.

Por último, el diseño no es puntuado muy alto, lo que nos dice que nuestro sistema deberá tener una mejora visual que sea atractiva para los usuarios.

En la gráfica ?? vemos cómo la administración y gestión deportiva a través de la aplicación no es ni lo mejor ni lo peor. No habría que aplicar mejoras respecto a lo existente, pero podemos añadir nuevas funcionalidades que faciliten el trabajo de los administradores.

Según la gráfica ??, la gestión de los partidos parece ser excelente. Aunque cabe recalcar que los árbitros utilizan diversas aplicaciones para gestionar los partidos: tanto para arbitrar un partido como para anotar los resultados. Por ello existe la necesidad de hacer un sistema unificado para todos los interesados.

La gráfica B.4 nos confirma la necesidad del sistema multiplataforma al observar el uso de tablets por parte de los árbitros para rellenar el acta de los partidos.

En la gráfica ?? vemos como la opinión respecto a la gestión de equipos es muy baja. Esto nos lleva a deducir que puede ser interesante añadir funcionalidades para mejorar la interactividad y participación de los usuarios a la hora de gestionar los equipos deportivos.

Nota final y necesidad de un nuevo sistema

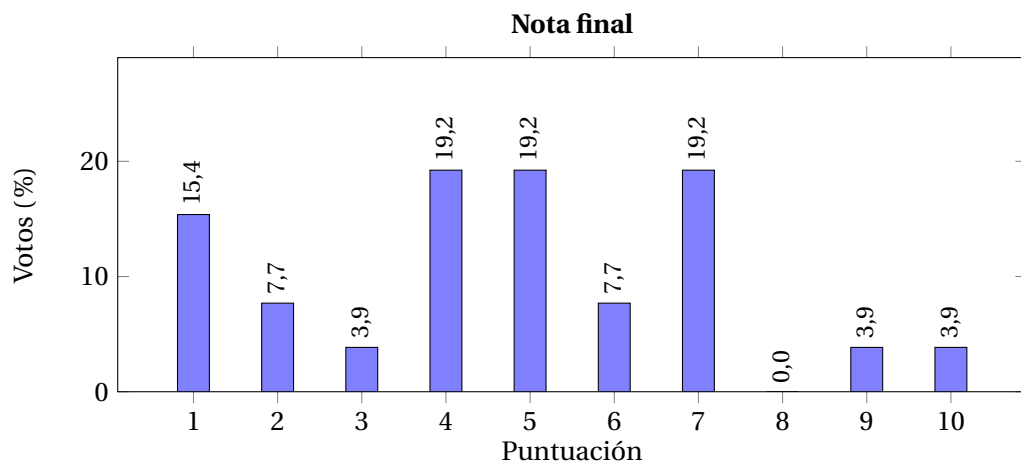


Figura 3.3: Nota final

En las puntuaciones finales (Ver 3.3), observamos un patrón en el que destacan los valores bajos y donde los valores altos son insignificantes en comparación. Esto nos da el veredicto final y nos describe el descontento generalizado que hay con la aplicación.

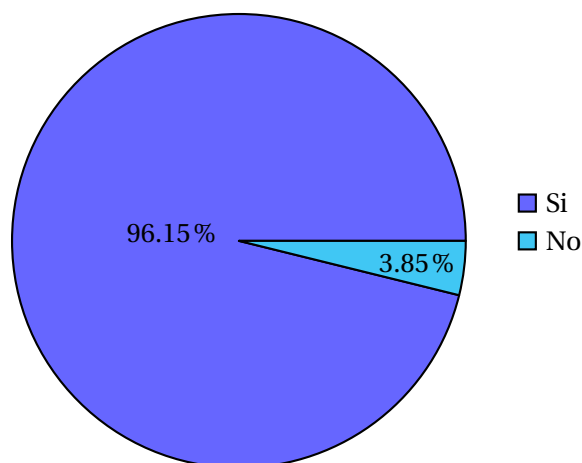


Figura 3.4: ¿Te gustaría una nueva aplicación que incluya nuevas y mejores funcionalidades?

La gráfica 3.4 es la clave y el principal motivo del desarrollo de un nuevo sistema. Nos confirma que la gran mayoría de los usuarios buscan nuevas opciones que mejoren la experiencia que han tenido hasta ahora con el sistema actual.

Aspectos específicos

Ahora que sabemos que es necesario un nuevo sistema de gestión, con las siguientes cuestiones sabremos qué funcionalidades son necesarias para poder cumplir con las exigencias y carencias de los usuarios. Y, a partir de los resultados, seremos capaces de definir los requisitos funcionales del nuevo sistema.

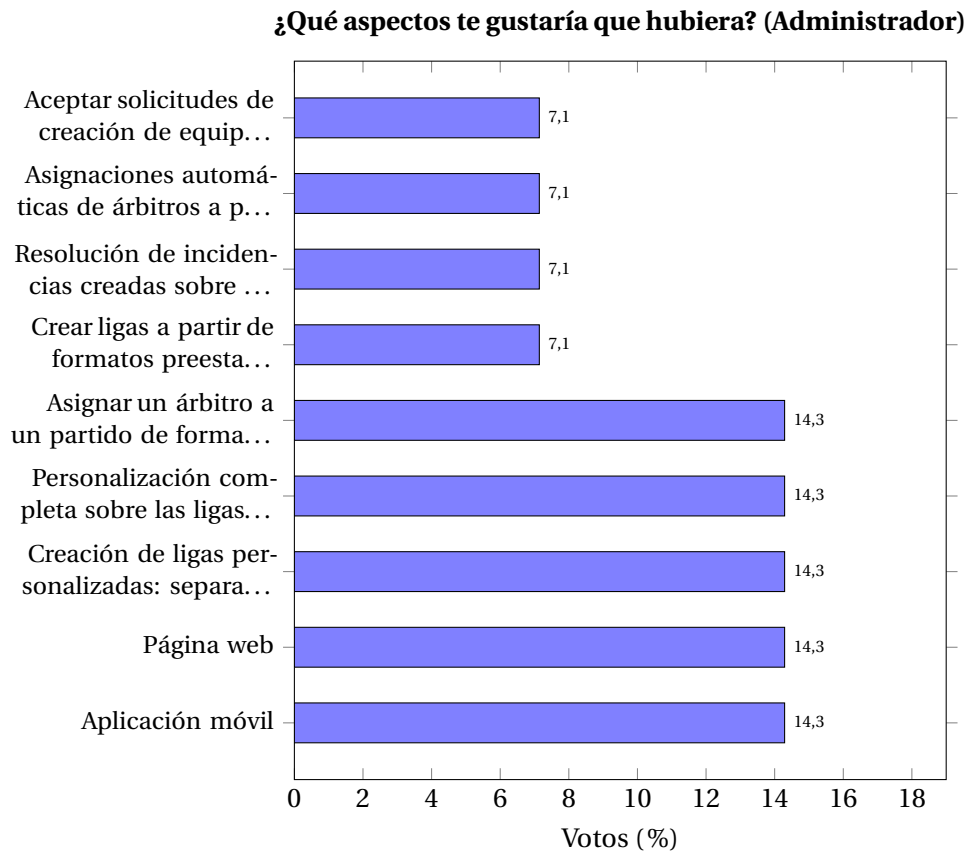


Figura 3.5: Nuevas funcionalidades: Administrador

Vemos como los administradores buscan una mayor versatilidad y facilidad a la hora de poder gestionar la liga desde cualquier dispositivo. Una gestión amena que no genere fricción con el usuario.

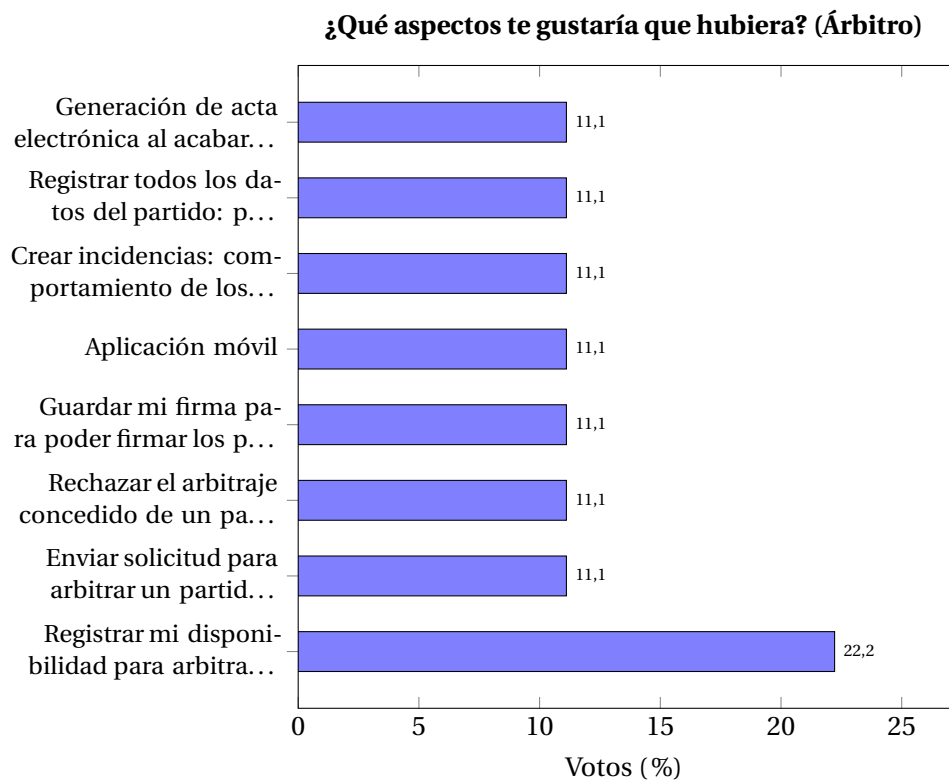


Figura 3.6: Nuevas funcionalidades: Árbitro

Mientras que los árbitros se centran principalmente en que puedan especificar su disponibilidad y que el sistema actúe acorde.

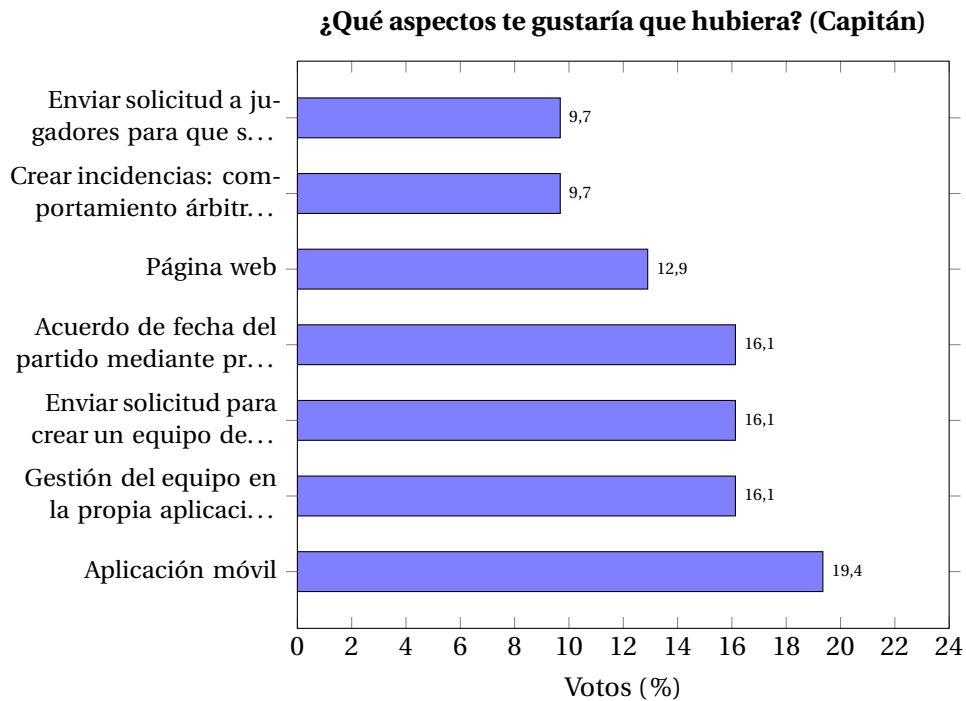


Figura 3.7: Nuevas funcionalidades: Capitán

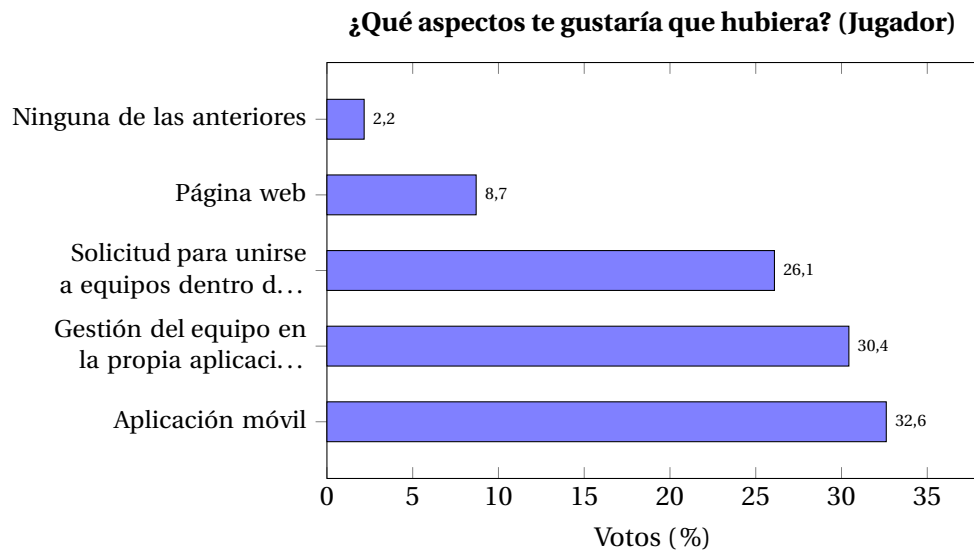


Figura 3.8: Nuevas funcionalidades: Jugador

Por último, tanto los capitanes como los jugadores buscan una mayor interacción con el sistema a la hora de gestionar los equipos.

Preguntas abiertas

Finalmente, con la sección de respuestas abiertas los usuarios comentan problemas que no han podido destacar con las secciones anteriores.

El análisis de las respuestas abiertas revela tres áreas principales de mejora que justifican el desarrollo de la nueva plataforma.

En primer lugar, destaca la falta de adaptación al voleibol. Los usuarios señalan de forma recurrente que el sistema actual está enfocado al fútbol, lo que provoca errores y confusión a la hora de registrar los resultados por sets, repartir los puntos y visualizar las clasificaciones.

En segundo lugar, se reportan notables deficiencias de rendimiento y usabilidad. Los encuestados mencionan lentitud general, fallos de carga en apartados como las notificaciones, errores en la actualización del estado de los partidos y dificultades técnicas para acceder a la aplicación en ciertos dispositivos móviles.

Por último, existe una clara demanda de mejores herramientas de gestión y comunicación. Las respuestas subrayan la necesidad de que los equipos puedan gestionar sus propias plantillas, recibir avisos de los próximos encuentros y contar con un sistema mucho más ágil para visualizar el calendario y asignar a los árbitros.

Para consultar el listado íntegro de las respuestas y los problemas específicos reportados por los usuarios, véase el B y B

Resumiendo, buscan un sistema multiplataforma, interactivo, abierto a participación y fácil de utilizar. Un sistema que refleje de forma mucho más clara el tipo de deporte que se está jugando y ofrezca funcionalidades que faciliten dicha gestión. Un sistema a prueba de fallos, eficiente, consistente y visualmente atractivo.

3.1.4 Reunión con la OUSIS

Al mismo tiempo que se empezó con la recogida de requisitos, se estableció contacto con la OUSIS. Habían escuchado que se estaba desarrollando este proyecto y decidieron contactar para aclarar ciertas dudas que tenían.

Tuvimos dos reuniones en las que no se hicieron ningún acta de reunión. En la primera, estuvo presente Raimon Massanet y consistió en una ronda de preguntas por parte de los responsables de la OUSIS: Joan Mercant Requena y Olalla Garcia Taibo. Fue necesario aclarar que las encuestas eran de carácter completamente anónimo y no se recogía ningún dato personal.

En la segunda reunión, Antonio Crespí presente, se estableció un contacto profesional para utilizar los resultados del proyecto como una base para implementar un nuevo sistema de gestión de actividades deportivas dentro de la UIB. Algunas de sus peticiones fueron: poder utilizar la aplicación para gestionar otras actividades que no sean ligas deportivas y extender el uso de la aplicación a más deportes, ya que en un principio la aplicación estaba enfocada únicamente a voleibol. Debido a esto, se intentó que todo el proyecto fuera capaz de soportar estos requisitos a futuro, pero sin alterar el alcance base del sistema.

3.1.5 Requisitos funcionales

A partir de los resultados de las encuestas y las reuniones con la OUSIS se han sintetizado los siguientes requisitos funcionales y no funcionales:

Cuadro 3.3: Requisitos funcionales del sistema

Código	Entidad	Descripción
RF-USR-1	Usuario	El sistema debe permitir a los usuarios poder crear una cuenta para registrar sus datos en el sistema.
RF-USR-2	Usuario	El sistema debe permitir a los usuarios registrar la siguiente información: nombre completo, email, contraseña, sexo, referencias de las distintas licencias arbitrales que pueda tener un usuario y foto de perfil.
RF-USR-3	Usuario	El sistema debe permitir que los usuarios puedan indicar el deporte al que corresponden sus distintas licencias arbitrales.
RF-USR-4	Usuario	El sistema debe permitir a los usuarios iniciar sesión con una cuenta ya creada dadas sus credenciales: nombre de usuario o email y contraseña.
RF-USR-5	Usuario	El sistema debe permitir a los usuarios con sesión iniciada crear ligas deportivas. El usuario que crea una liga se convierte automáticamente en administrador de la liga.
RF-USR-6	Usuario	El sistema debe permitir a los usuarios sin cuenta poder ver ligas ya creadas.
RF-USR-7	Usuario	El sistema debe permitir que los usuarios puedan almacenar su firma como una imagen que pueda ser reutilizada para firmar en los partidos en caso de ser capitán o árbitro.
RF-LIG-1	Liga	El sistema debe permitir crear ligas con la siguiente información: nombre, descripción, icono, banner, ubicación, fecha de inicio, fecha de fin y fecha máxima para inscribirse.
RF-LIG-2	Liga	El sistema debe permitir configurar los siguientes parámetros dentro de una liga: categoría (Masculina, Femenina o Mixta), número de integrantes femeninos mínimos en caso de que la categoría sea mixta, número mínimo y máximo de integrantes en cada equipo y duración de las jornadas en semanas.

Continúa en la página siguiente

Cuadro 3.3 – continuación de la página anterior

Código	Entidad	Descripción
RF-LIG-3	Liga	El sistema debe permitir implementar un sistema de roles dentro de las ligas: Administrador, Árbitro, Capitán y Jugador.
RF-LIG-4	Liga	El sistema debe permitir a los usuarios con sesión iniciada unirse a ligas creadas. Al unirse se convierte en Jugador dentro de esa liga.
RF-LIG-5	Liga	El sistema debe permitir que los participantes de una liga puedan crear incidencias dentro de esta. Las incidencias deben ir acompañadas de una descripción.
RF-LIG-6	Liga	El sistema debe permitir que los administradores de una liga puedan expulsar a los participantes de esta.
RF-LIG-7	Liga	El sistema debe permitir que una liga esté conformada por distintas fases.
RF-LIG-8	Liga	El sistema debe permitir que un usuario pueda elegir ver únicamente las ligas en las que está participando.
RF-EQU-1	Equipo	El sistema debe permitir a los usuarios dentro de una liga crear y enviar solicitudes a los administradores para poder crear equipos dentro de la liga.
RF-EQU-2	Equipo	El sistema debe poder permitir crear equipos con la siguiente información: nombre, iniciales, descripción, lema, color primario, color secundario y escudo.
RF-EQU-3	Equipo	El sistema debe permitir al administrador de una liga aceptar o rechazar solicitudes de creación de equipo. En caso de ser rechazada se debe poder incluir un mensaje con la justificación pertinente.
RF-EQU-4	Equipo	El sistema debe permitir que, al aceptar una solicitud de creación de equipo, el solicitante se convierta automáticamente en el capitán del equipo solicitado.

Continúa en la página siguiente

Cuadro 3.3 – continuación de la página anterior

Código	Entidad	Descripción
RF-EQU-5	Equipo	El sistema debe permitir que los jugadores que no tengan equipo puedan enviar una solicitud al capitán de un equipo para poder unirse a este.
RF-EQU-6	Equipo	El sistema debe permitir que los capitanes puedan aceptar o rechazar las solicitudes de unión de su equipo.
RF-EQU-7	Equipo	El sistema debe permitir que los capitanes de los equipos puedan enviar solicitudes a jugadores sin equipo para que se unan a este.
RF-EQU-8	Equipo	El sistema debe permitir que los jugadores puedan aceptar o rechazar las solicitudes para unirse a un equipo.
RF-EQU-9	Equipo	El sistema debe permitir que los capitanes de un equipo puedan expulsar a los jugadores de este.
RF-EQU-10	Equipo	El sistema debe permitir que los jugadores puedan elegir un número de dorsal dentro de su equipo que no se haya elegido antes.
RF-EQU-11	Equipo	El sistema debe permitir que el capitán de un equipo pueda eliminar el equipo que lidera.
RF-ROL-1	Rol	El sistema debe permitir que los participantes de una liga puedan tener uno o más roles asignados.
RF-ROL-2	Rol	El sistema debe permitir que el creador de una liga pueda asignar el rol de Administrador a otros participantes de esta. De este modo se puede repartir el trabajo que conlleva gestionar la liga.
RF-ROL-3	Rol	El sistema debe permitir que los jugadores de una liga puedan crear solicitudes para que los administradores les puedan asignar el rol de Árbitro en dicha liga.

Continúa en la página siguiente

Cuadro 3.3 – continuación de la página anterior

Código	Entidad	Descripción
RF-ROL-4	Rol	El sistema debe permitir que los administradores rechacen o acepten las solicitudes de los jugadores para convertirse en árbitros. Dicha respuesta y solicitud cuenta con la misma información que las solicitudes para crear equipos.
RF-ROL-5	Rol	El sistema debe permitir que los administradores puedan retirar el rol de Árbitro a los participantes árbitros que vea conveniente.
RF-ROL-6	Rol	El sistema debe permitir que los administradores puedan eliminar un equipo.
RF-FAS-1	Fase	El sistema debe permitir configurar cada fase con la siguiente información: nombre, fecha de inicio, fecha de fin, formato y su orden dentro de la secuencia de fases.
RF-FAS-2	Fase	El sistema debe permitir que cada fase pueda ser uno de los siguientes formatos: Torneo y Clasificatorio. En el formato Clasificatorio se debe especificar el número de ganadores que tendrá. Podrían añadirse nuevos formatos en un futuro.
RF-FAS-3	Fase	El sistema debe permitir que cada fase esté compuesta por una o más Rondas.
RF-RON-1	Ronda	El sistema debe permitir que para cada Ronda se pueda establecer el día en que empieza dicha Ronda.
RF-RON-2	Ronda	El sistema debe permitir que en cada ronda se celebren diversos partidos.
RF-RON-3	Ronda	El sistema debe permitir que los usuarios con rol de Administrador de una liga establezcan la disponibilidad de la ubicación de juego para cada Ronda.
RF-DIS-1	Disponibilidad	El sistema debe permitir que la disponibilidad de una ronda se establezca como un conjunto de franjas horarias.
RF-DIS-2	Disponibilidad	El sistema debe permitir que en cada franja horaria se pueda especificar el día, la hora y la duración de la franja.

Continúa en la página siguiente

Cuadro 3.3 – continuación de la página anterior

Código	Entidad	Descripción
RF-DIS-3	Disponibilidad	El sistema debe permitir que los administradores de una liga puedan establecer la disponibilidad de la ubicación de juego para cada ronda.
RF-DIS-4	Disponibilidad	El sistema debe permitir que cada usuario, en especial los capitanes y los árbitros, puedan establecer su disponibilidad para cada ronda respecto a la disponibilidad establecida por los administradores para la ubicación de juego.
RF-PAR-1	Partido	El sistema debe permitir que los partidos se generen de forma automática dependiendo del formato de la Fase.
RF-PAR-2	Partido	El sistema debe permitir que en cada partido se enfrenten un equipo local y un equipo visitante.
RF-PAR-3	Partido	El sistema debe permitir que las asignaciones de los equipos que se enfrentan en los partidos se pueda hacer de forma automática o manual.
RF-PAR-4	Partido	El sistema debe permitir que la fecha de un partido se establezca mediante un sistema de negociación entre los capitanes de los equipos que deben enfrentarse.
RF-PAR-5	Partido	El sistema debe permitir que los administradores de una liga puedan fijar la fecha de un partido que no se haya fijado.
RF-PAR-6	Partido	El sistema debe permitir que a cada partido se pueda asignar un árbitro. El árbitro se puede asignar de forma manual por parte de los Administradores o de manera automática.

Continúa en la página siguiente

Cuadro 3.3 – continuación de la página anterior

Código	Entidad	Descripción
RF-PAR-7	Partido	El sistema debe permitir que la forma automática para asignar un árbitro a un partido se haga mediante un algoritmo que, para un árbitro con una cierta disponibilidad, se le asigne un partido cuya fecha coincida con alguna de las franjas disponibles del árbitro. El algoritmo también debe tener en cuenta la cantidad de partidos arbitrados por cada árbitro para poder repartir el arbitraje de manera equitativa.
RF-PRO-1	Propuesta	El sistema debe permitir que el proceso de negociación de la fecha de un partido consista en lo siguiente: el capitán del equipo local crea y envía una primera propuesta con la franja horaria disponible deseada al capitán del equipo visitante. El capitán del equipo visitante puede decidir aceptar o rechazar la propuesta, si la rechaza debe enviar una contrapropuesta. Este proceso se repite hasta que alguno de los dos acepte la propuesta.
RF-RES-1	Resultado	El sistema debe permitir que se puedan guardar un Resultado para cada partido.
RF-RES-2	Resultado	El sistema debe permitir que cada resultado contenga la siguiente información: puntuación del equipo local, puntuación del equipo visitante, observaciones encontradas durante el partido y firmas de los árbitros y capitanes antes y después del partido.
RF-RES-3	Resultado	El sistema debe permitir que cada capitán y árbitro pueda firmar el acta de un partido desde su propio dispositivo.
RF-RES-4	Resultado	El sistema debe permitir que un partido se divida en distintos Periodos.
RF-RES-5	Resultado	El sistema debe permitir que cada periodo guarde la siguiente información: orden numérico del periodo dentro del partido, tipo de periodo según el deporte (set, mitad, cuarto, etc.) y puntuación de los dos equipos para ese periodo.

Continúa en la página siguiente

3. ANÁLISIS Y DISEÑO DEL SISTEMA

Cuadro 3.3 – continuación de la página anterior

Código	Entidad	Descripción
RF-PER-1	Periodo de Partido	El sistema debe permitir que cada Periodo contenga: la puntuación del equipo local y del equipo visitante para ese periodo, orden numérico del periodo dentro del partido y tipo de periodo según el deporte.
RF-PER-2	Periodo de Partido	El sistema debe permitir que cada periodo pueda tener Eventos.
RF-EVE-1	Evento de Partido	El sistema debe permitir que para cada evento se guarde la siguiente información: el tiempo en el que se genera respecto al inicio del partido, puntuaciones del equipo local y visitante para ese periodo y qué equipo ha sido artífice de dicho evento.
RF-EVE-2	Evento de Partido	El sistema debe permitir que cada evento pueda ser uno de tres tipos distintos: sustitución, sanción o tiempo muerto. Para la sustitución se debe especificar los dos jugadores que se intercambian. En la sanción se debe especificar una descripción y el tipo de sanción. Para los tiempos muertos se debe especificar la duración de dicho tiempo.

Podemos apreciar mejor los requisitos funcionales mediante el diagrama de casos de uso presente en el apéndice Diagrama de casos de uso

Cuadro 3.4: Requisitos no funcionales del sistema

Código	Tipo	Descripción
RNF-USA-1	Usabilidad	La aplicación móvil debe permitir a un árbitro registrar eventos y cerrar un acta en un máximo de 3 a 5 clics desde la pantalla principal.
RNF-USA-2	Usabilidad	La interfaz debe contar con un contraste adecuado para garantizar la legibilidad en entornos exteriores (pistas al aire libre con mucha luz).

Continúa en la página siguiente

Cuadro 3.4 – continuación de la página anterior

Código	Tipo	Descripción
RNF-SEG-1	Seguridad	El acceso a los endpoints críticos (como forzar un horario o modificar roles) debe estar protegido mediante autenticación por token (JWT) y validación estricta de roles y atributos: ABAC.
RNF-SEG-2	Seguridad	Las contraseñas de los usuarios no se almacenarán en texto plano bajo ninguna circunstancia; se utilizará un algoritmo de 'hashing' fuerte (como 'bcrypt').
RNF-SEG-3	Seguridad	Sanitización de entradas: Todos los datos enviados desde la aplicación móvil hacia la API deben ser validados y sanitizados para prevenir ataques de inyección SQL (gestionado a través del ORM de <i>Spring JPA</i>) y ataques XSS.
RNF-DEP-1	Fiabilidad	El sistema debe garantizar la atomicidad de las operaciones críticas (como el guardado del acta final). Si la conexión se pierde a mitad del envío, la base de datos debe revertir los cambios para evitar estados inconsistentes (actas a medias).
RNF-EFI-1	Eficiencia(Espacio)	Las imágenes subidas al sistema (como firmas de capitanes, fotos de perfil o logos de equipos) deben comprimirse para no superar un peso máximo de 200KB por imagen.
RNF-EFI-2	Eficiencia(Rendimiento)	Las consultas de alta frecuencia, como la visualización de la clasificación, deben resolverse en menos de 200 ms desde el back-end, apoyándose en la base de datos optimizada (vistas materializadas).
RNF-EFI-3	Eficiencia(Rendimiento)	La aplicación móvil debe arrancar y mostrar la pantalla de inicio (Dashboard) en menos de 3 segundos en condiciones de red estándar (4G/Wifi).

Continúa en la página siguiente

Cuadro 3.4 – continuación de la página anterior

Código	Tipo	Descripción
RNF-OPE-1	Operacional	Las constantes del sistema de competición (como puntos por victoria, duración de jornadas) deben ser configurables desde la base de datos sin necesidad de recompilar el código del servidor.
RNF-OPE-2	Pila Tecnológica	Para garantizar la homogeneidad y mantenibilidad del sistema, el back-end debe desarrollarse estrictamente en Kotlin con Spring Boot (exponiendo API REST) y el front-end móvil en React Native con Expo. La persistencia de datos relacionales recaerá exclusivamente en PostgreSQL, mientras que la persistencia de datos no relacionales recaerá en un servicio Object Storage - S3 implementado con MinIO.
RNF-OPE-3	Contenerización	El entorno del servidor debe estar empaquetado mediante contenedores de Docker para asegurar que la aplicación funcione de manera idéntica tanto en la máquina local de desarrollo como en el servidor de producción.
RNF-DES-1	Desarrollo	Control de Versiones: Todo el código fuente del proyecto (front-end, back-end y scripts de base de datos) debe estar centralizado y versionado utilizando Git, alojado en un repositorio en la nube como GitHub.
RNF-DES-2	Desarrollo	Trunk-Based Development (Etiquetas): El equipo de desarrollo debe seguir un flujo de trabajo estructurado. Existirá una rama 'main' que contendrá todo el código desarrollado. Sobre esta rama 'main' se irán creando etiquetas para marcar versiones funcionales y desplegables del sistema.

Continúa en la página siguiente

Cuadro 3.4 – continuación de la página anterior

Código	Tipo	Descripción
RNF-DES-3	Desarrollo	Pruebas Automatizadas (Pruebas Unitarias): La lógica de negocio crítica en Kotlin (por ejemplo, el cálculo de los puntos de la liga o la máquina de estados de los partidos) debe estar cubierta por pruebas unitarias independientes de la base de datos.
RNF-DES-4	Desarrollo	Pruebas Automatizadas (Pruebas de Integración): Se deben incluir pruebas que verifiquen el correcto funcionamiento de los endpoints principales y sus consultas contra una base de datos de prueba (por ejemplo, usando "Testcontainers." o una base de datos en memoria).
RNF-DES-5	Desarrollo	Gestión de variables de entorno: Ninguna credencial, token secreto o contraseña de base de datos debe subirse al repositorio de código. Todas las configuraciones sensibles deben inyectarse mediante variables de entorno (archivos .env en React Native y application.yml parametrizado en Spring Boot).
RNF-DES-6	Desarrollo	El diseño de la API REST debe estar documentado mediante el estándar Open API (Swagger), actuando como único contrato y fuente de la verdad para sincronizar el trabajo entre el front-end y el back-end.
RNF-DES-7	Desarrollo	El código fuente del back-end debe seguir los principios de separación por capas (Controlador, Servicio, Repositorio) para asegurar que la lógica de negocio esté aislada y sea fácilmente verificable. Además, debe aplicarse una "separación vertical" por entidades para mantener dichas funcionalidades claramente separadas.

3.2 Diseño

Definiremos el diseño en dos niveles distintos: la distribución e interacción de los distintos componentes y cómo se va a diseñar cada componente a nivel interno.

3.2.1 Arquitectura del sistema

Las siguientes imágenes que representan los tres primeros niveles de un modelo C4 explican en detalle la arquitectura establecida para Sports League:

El nivel 1 muestra cómo interactúan los distintos tipos de usuarios con la aplicación. Además, muestra la relación del servicio de autenticación de la UIB con nuestro sistema.

En el nivel 2 podemos observar cómo los distintos componentes interactúan entre ellos: la interfaz gráfica se comunica con el servidor back-end para la consulta y envío de datos a través de peticiones HTTP, mientras que el servidor back-end se comunica con los distintos almacenamientos a través de sus API específicas.

El nivel 3 indica cómo se ha estructurado el back-end: el núcleo del sistema. Podemos observar dos versiones diferentes para este nivel. Para poder cumplir con el requisito no funcional [...] se ha decidido utilizar dos tipos de arquitectura de software diferentes: una arquitectura de tres capas simple para los módulos que engloban entidades con ciclos de vida simple (creación, visualización y actualización); mientras que para módulos de entidades con ciclos de vida más complejos, (seguridad, autenticación, mantenimiento de sub recursos, cambios y actualizaciones complejas de varios pasos transaccionales, etc.) se aplica la arquitectura hexagonal o arquitectura de puertos y adaptadores.

En el caso de la arquitectura de tres capas, vemos como hay un flujo de datos desde los controladores que reciben las peticiones del front-end aceptadas por el filtro de seguridad y autenticación, pasando por los servicios sobre los que se delega la lógica de negocio y acabando en los repositorios, encargados de la persistencia y obtención de los datos.

Por otro lado, en la arquitectura hexagonal, vemos como las peticiones llegan a la capa de infraestructura, pasando del filtro de seguridad al adaptador de entrada correspondiente, este procede a llamar al puerto necesario para delegar la lógica de negocio a la capa de aplicación y dicho caso de uso utiliza los repositorios implementados por los adaptadores de salida para comunicarse con las bases de persistencia de datos.

3.2.2 Diseño del sistema

3.2.3 Diseño de la base de datos

El diseño de la Base de Datos (BD) ha sido un derivado directo de los requisitos funcionales y muestra cómo las entidades se relacionan entre ellas a partir de los requisitos. Se ha utilizado la herramienta de draw.io para crear un diagrama de clases UML que muestra la estructura de la BD. En el enlace Diagrama UML - draw.io podemos ver el diagrama más detalladamente.

En el diagrama se pueden apreciar: las clases definidas con sus atributos correspondientes y las relaciones que relacionan dichas clases. Cabe aclarar que se pueden ver algunos de los nombres de clases repetidos junto a un asterisco '*'. Dicho formato indica que la clase ya ha sido definida y que simplemente se usa como una 'referencia'.

Se ha decidido aplicar este formato por claridad visual. De esta forma evitamos que las relaciones que conectan dos clases muy separadas no estorben en la lectura del diagrama.

Aparte de la vista general del diagrama resultaría interesante destacar dos zonas en concreto: una por ser la pieza fundamental del sistema y la otra por resultar en un ejemplo de normalización muy claro.

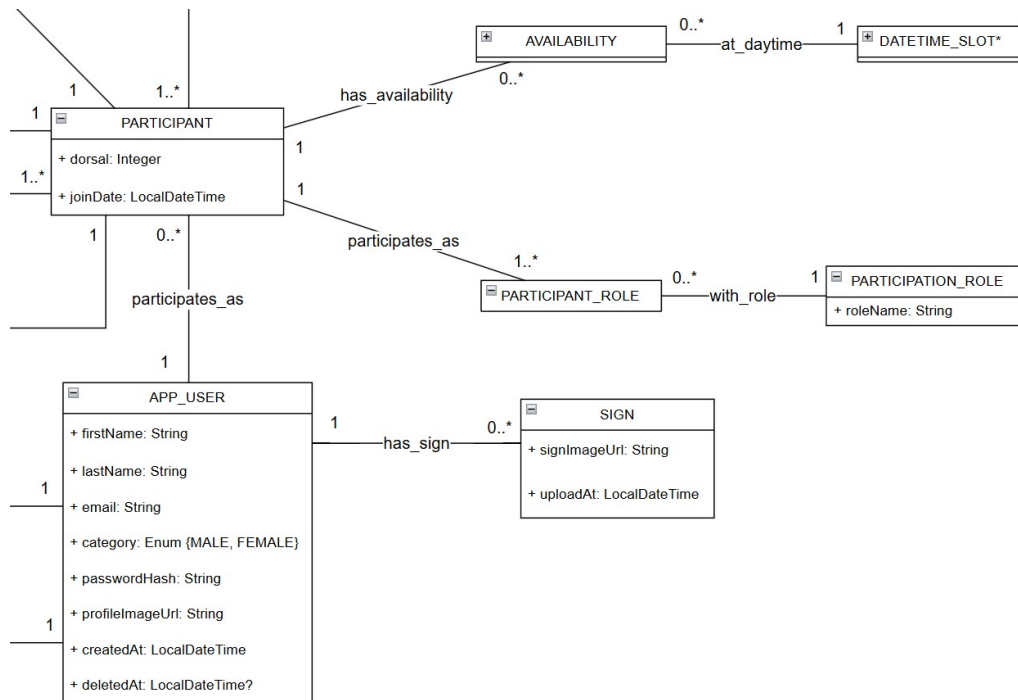


Figura 3.9: Clase participante

En la primera zona (Ver Figura 3.9), la pieza central del sistema, se aprecia la clase Participante: una clase asociativa que relaciona de manera única un usuario con una liga. Esta clase existe porque un usuario puede estar inscrito en varias ligas a la vez y una misma liga puede tener diversos usuarios simultáneos. Esta clase también se relaciona con otra clase asociativa que es la que permite establecer el sistema de roles dentro de una liga. Gracias a estas relaciones conseguimos que un mismo usuario tenga distintos roles dependiendo de la liga. Y, este punto, es el que convierte el sistema en uno que necesita manejar la multitenencia. Además, se puede ver la clase *Availability* que es la que permite que los árbitros puedan registrar su disponibilidad para arbitrar y que el sistema asigne los partidos de forma automática.

La segunda zona destacada (Ver Figura 3.10) se relaciona con la parte encargada de gestionar los resultados de los partidos. También, se encarga de la gestión de todos los datos de un partido necesarios para poder generar el acta del partido. Esta zona ha resultado en un buen ejemplo de normalización, ya que, en un principio, se tenía pensado agrupar toda la información del partido en un atributo de la clase Resultado de tipo JSONB.

Se consideró esta primera opción por la naturaleza cambiante y dinámica de la información presente en las actas oficiales de la FVIB (Ver ejemplo en el enlace: Acta

3. ANÁLISIS Y DISEÑO DEL SISTEMA

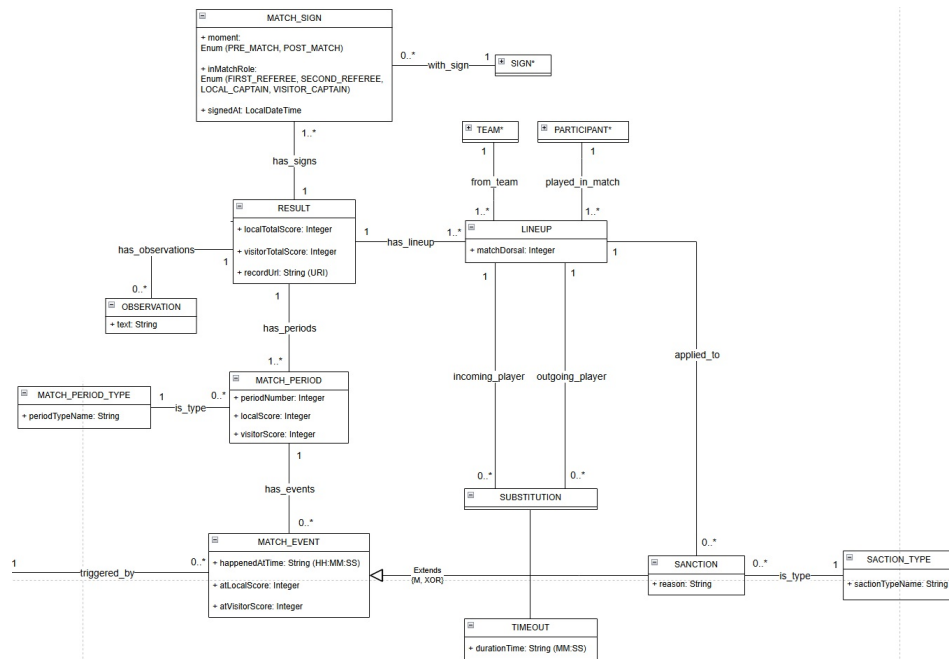


Figura 3.10: Resultado y acta

oficial). El problema de esta opción era que desnormalizaba en gran manera los datos de un partido, lo cual podría resultar en deuda técnica para manejar dichos datos a futuro. Debido a este problema, se optó por la segunda y opción final: analizar en profundidad todos los aspectos de una acta oficial y modelar estos aspectos manteniendo la tercera forma normal.

Como se puede ver en la Figura [...] tenemos un conjunto de clases relacionadas que modelan todos los datos necesarios para generar un acta oficial. Se podría argumentar que la inclusión de algunos atributos introduce redundancia en el modelo, ya que las puntuaciones de los equipos almacenadas en la clase *Resultado* pueden obtenerse sumando los puntos de los distintos periodos del partido. Sin embargo, se trata de una redundancia controlada y deliberada: los atributos se registran una única vez al cerrar el acta y permiten consultar directamente el resultado global del partido. Esta decisión ofrece dos ventajas. En primer lugar, evita recalcular las puntuaciones cada vez que se consulta el resultado. En segundo lugar, mejora el rendimiento al mostrar listados de partidos, ya que no es necesario sumar los puntos de los periodos de cada partido visible en pantalla.

3.2.4 Diseño de la interfaz

Es necesario pensar en cómo los usuarios pueden interactuar con el sistema para cumplir con requisitos, es decir, necesitamos definir una User Experience, Experiencia de Usuario (UX) que sea, según las respuestas de los usuarios y requisitos funcionales: manejable y fácil de utilizar.

Para el diseño de la interfaz se ha optado por usar la herramienta de Figma al ser un estándar en el mundo del diseño sin coste. La versión preliminar de la interfaz se puede ver a través del siguiente enlace: Mockup en Figma

Cabe tener en cuenta que este diseño tan solo muestra la forma en la que los usuarios interactuarán con la interfaz y cómo se les mostrará la información. Todo lo relacionado con el diseño visual, estilos, paletas de colores, logotipos y demás se delegará a un agente de generación de código que será el encargado de darle estilo la interfaz.

3.2.5 Diseño del contrato API

El desarrollo del sistema se ha enfocado como un proceso API-first. Lo que quiere decir, que el primer paso ha sido definir un contrato API que funciona como única fuente de la verdad y del que se derivan todos los Data Transfer Object, Objeto de Transferencia de Datos (DTO) utilizados tanto en el back-end como en el front-end, para estandarizar la estructura de los mensajes que permiten la comunicación entre estos dos servicios. Para redactar el contrato se ha hecho uso del estándar Open API ampliamente utilizado en la industria del desarrollo.

La definición del contrato se ha hecho de forma que sea una representación casi idéntica del modelo de datos (Ver 3.2.3). Se han separado los esquemas según las clases pero agrupándolas según sus relaciones. Además, se han creado esquemas enumerados para poder definir de forma concreta y consistente algunos atributos de las clases como lo puede ser los estados de una petición, una liga

El contenido completo del contrato API se puede encontrar en el repositorio correspondiente.

3.2.6 Control de versiones

Para gestionar el control de versiones de los componentes del sistema se ha decidido utilizar Git. Es la herramienta que más se usa dentro de la industria y la más fácil de utilizar para un equipo de desarrollo de una sola persona, como es el caso de este proyecto.

Se han creado tres repositorios, uno para cada componente principal del sistema: back-end, front-end y contrato API. Se ha optado por esta separación, ya que de esta manera separamos el sistema según el ámbito: definición, procesamiento y presentación de los datos. Un aspecto interesante es que el repositorio del contrato API existe como submódulo de los otros dos. Gracias a esto, ambos componentes tienen una vía directa para actualizar el contrato en caso de ser necesario y consultar los cambios que haya efectuado el otro componente. De esta forma, ambos componentes están al día

ESTADO DEL ARTE EN EL ÁMBITO TECNOLÓGICO

Para implementar este tipo de sistemas requerimos de tecnologías que nos permitan cumplir con todos los requisitos especificados y el diseño mostrado en el capítulo anterior. Para cada componente veremos: una breve explicación de la necesidad de dicho componente, listado de las distintas alternativas ideales que existen para dicho componente, una tabla comparativa para distintos aspectos técnicos y la tecnología seleccionada con su correspondiente justificación.

4.1 Servidor back-end

Para el servidor central que gestiona la lógica de negocio y expone la API REST, se requiere un sistema fuertemente tipado, escalable y con un ecosistema maduro de dependencias.

Evaluación de alternativas

Spring Boot (Kotlin)

Framework empresarial maduro que destaca por su robusto sistema de inyección de dependencias y módulos integrados (Spring Security, Data JPA). El uso del lenguaje Kotlin aporta un tipado estático moderno, inmutabilidad y seguridad contra referencias nulas (*Null-safety*) de forma nativa.

Node.js (NestJS)

Entorno de ejecución asíncrono basado en TypeScript. Ofrece tiempos de arranque muy rápidos y un ecosistema masivo de paquetes. Su arquitectura basada en eventos es ideal para alta concurrencia de entrada/salida, aunque delega gran parte de la estructura empresarial a librerías de terceros.

Ktor

Framework asíncrono ligero creado por JetBrains para Kotlin. Ofrece un alto rendimiento mediante el uso de corrutinas y un diseño altamente modular, pero carece de las soluciones empresariales completas que proporciona el ecosistema Spring.

Comparativa técnica

En la Tabla 4.1 se resumen las características fundamentales de cada entorno.

Tecnología Criterio	Spring (Kotlin)	Boot	NestJS (TypeS- cript)	Ktor (Kotlin)
Paradigma	Multihilo / Síncrono		Basado en eventos	Asíncrono (Corrutinas)
Seguridad de tipos	Estricta (Nativa)		Alta (Transpilado)	Estricta (Nativa)
Ecosistema	Muy maduro		Masivo (NPM)	En crecimiento
Arquitectura	Altamente estructurada		Modular	Minimalista

Cuadro 4.1: Comparativa de frameworks para el desarrollo del back-end.

Justificación de la elección

Se ha seleccionado **Spring Boot con Kotlin** por los siguientes motivos técnicos:

- **Diseño arquitectónico:** Su robusta inyección de dependencias facilita la implementación estricta de la Arquitectura Hexagonal, permitiendo un desacoplamiento real entre los puertos y sus adaptadores.
- **Mitigación de riesgos:** Contar con experiencia previa en el ecosistema Spring reduce los tiempos de configuración y asegura una implementación estable del sistema de seguridad e interacciones con la base de datos.
- **Modernización técnica:** La adopción de Kotlin representa un reto académico de alto valor, permitiendo escribir un código de dominio más conciso y expresivo frente a lenguajes tradicionales como Java.

4.2 Base de datos

Para dar soporte a la persistencia del sistema se requiere un Sistema de Gestión de Bases de Datos Relacionales (SGBDR) que garantice el cumplimiento de las propiedades Atomicidad, Consistencia, Aislamiento y Durabilidad (ACID).

Evaluación de alternativas

PostgreSQL

Sistema objeto-relacional de código abierto con adherencia estricta a los estándares SQL. Destaca por su alta extensibilidad, soportando tipos de datos avanzados y manipulación nativa de estructuras semiestructuradas mediante JSONB. Su modelo de

control de concurrencia multiversión (MVCC) optimiza el rendimiento en entornos de alta carga transaccional.

MySQL

Uno de los motores relacionales más extendidos, basado en el motor de almacenamiento *InnoDB* para garantizar soporte ACID. Sobresale por su facilidad de despliegue, curva de aprendizaje inicial baja y alto rendimiento en operaciones de lectura simples.

MariaDB

Bifurcación comunitaria de MySQL, compatible a nivel de binarios, pero con optimizadores de consultas más avanzados y un pool de hilos nativo para mejorar el manejo de la concurrencia.

Comparativa técnica

En la Tabla 4.2 se resumen las principales características técnicas evaluadas.

Tecnología Criterio	PostgreSQL	MySQL (InnoDB)	MariaDB
Modelo	Objeto-relacional	Relacional	Relacional
Cumplimiento SQL	Estricto	Medio-alto	Medio-alto
Consultas complejas	Excelente (CTE, sub consultas)	Adecuado	Bueno
Tipos avanzados	Arrays, JSONB indexable, UUID	JSON básico	JSON básico

Cuadro 4.2: Comparativa de motores de bases de datos relacionales.

Justificación de la elección

El modelo de datos del dominio deportivo presenta una fuerte necesidad de integridad referencial, lo que descarta arquitecturas no relacionales. Se ha seleccionado **PostgreSQL** por los siguientes motivos:

- **Integridad de datos:** Permite aplicar restricciones complejas directamente en la base de datos, asegurando la consistencia del dominio.
- **Flexibilidad estructural:** El uso de JSONB permite almacenar metadatos variables sin perder la rigidez relacional del resto del esquema.
- **Robustez a largo plazo:** Garantiza un rendimiento superior en consultas analíticas complejas (como la generación de clasificaciones o estadísticas).

Para gestionar la evolución del esquema y asegurar la reproducibilidad del entorno, se ha integrado **Flyway**, herramienta que permite aplicar un control de versiones estricto sobre las migraciones *.sql*.

4.3 Almacenamiento de objetos (Object Storage)

Para la gestión de archivos no estructurados generados por el sistema, como imágenes de perfil o documentos PDF (actas deportivas), se requiere un servicio de almacenamiento de objetos independiente de la base de datos para no degradar el rendimiento transaccional.

Evaluación de alternativas

MinIO

Servidor de almacenamiento de objetos de alto rendimiento, 100 % de código abierto, diseñado para infraestructuras locales o privadas. Es totalmente compatible con la API de Amazon S3, permitiendo una integración estandarizada.

Amazon S3

El estándar de facto en la industria para el almacenamiento de objetos en la nube pública. Ofrece una disponibilidad hasta del 99.99999999 % y escalabilidad ilimitada, delegando la gestión de la infraestructura al proveedor.

Firebase Storage

Servicio gestionado de Google orientado a integraciones rápidas en aplicaciones móviles y web, fuertemente acoplado al ecosistema de Firebase y a sus reglas de seguridad.

Comparativa técnica

En la Tabla 4.3 se analizan las soluciones basándonos en las necesidades del entorno de desarrollo y producción del proyecto.

Tecnología Criterio	MinIO	Amazon S3	Firebase Storage
Despliegue	Local / Autoalojado (Docker)	Nube gestionada (AWS)	Nube gestionada (GCP)
Compatibilidad API	Estándar S3	Estándar S3	Propietaria
Soberanía de datos	Total (Bajo control propio)	Cedida al proveedor	Cedida al proveedor
Gestión de Infraestructura	Responsabilidad del desarrollador	Automática	Automática

Cuadro 4.3: Comparativa de soluciones de almacenamiento de objetos.

Justificación de la elección

Se ha seleccionado **MinIO** como motor de almacenamiento de objetos por las siguientes razones estratégicas:

- **Soberanía y control:** Al desplegarse mediante contenedores de forma local, garantiza la privacidad total de los documentos y evita la dependencia (vendor lock-in) frente a los cambios de políticas de precios de terceros.
- **Entorno de desarrollo aislado:** Permite a cualquier desarrollador levantar el ecosistema completo sin necesidad de configurar credenciales en la nube ni incurrir en costes operativos durante la fase de creación.
- **Compatibilidad futura:** Al implementar exactamente la misma API que Amazon S3, el código desarrollado en el back-end es completamente agnóstico. Si en el futuro el proyecto requiere escalar a la nube pública, la migración a AWS S3 se podría realizar modificando únicamente variables de entorno, sin reescribir la lógica de la aplicación.

4.4 Aplicación front-end

Para la interfaz de usuario, se requiere una solución multiplataforma que evite la duplicidad de bases de código para web, iOS y Android, garantizando un rendimiento fluido y un acceso estandarizado a las funcionalidades nativas del dispositivo.

Evaluación de alternativas

React Native con Expo

Framework que renderiza componentes nativos reales comunicándose a través de un puente con el hilo de JavaScript/TypeScript. Expo abstrae la configuración de los entornos nativos, ofreciendo empaquetado rápido, recarga en caliente y gestión simplificada de despliegues.

Flutter

Framework de Google basado en el lenguaje Dart que dibuja su propia interfaz mediante el motor gráfico Skia/Impeller. Ofrece un rendimiento visual sobresaliente en animaciones complejas, pero requiere adoptar un ecosistema completamente distinto.

Ionic / Capacitor

Solución híbrida que ejecuta una aplicación web estándar (HTML/CSS/JS) dentro de un navegador interno (*WebView*) embebido en la aplicación nativa. Destaca por su facilidad de desarrollo web, pero sufre limitaciones de rendimiento en interfaces pesadas.

Comparativa técnica

En la Tabla 4.4 se detalla la comparativa técnica de las tecnologías evaluadas.

4. ESTADO DEL ARTE EN EL ÁMBITO TECNOLÓGICO

Tecnología Criterio	React Native + Expo	Flutter	Ionic
Renderizado UI	Componentes Nativos	Motor gráfico propio	WebView
Lenguaje	TypeScript / JavaScript	Dart	TypeScript / Web
Acceso a Hardware	Mediante puente (JSI)	Integración compilada	Plug-ins (Capacitor)
Desarrollo ágil	Muy alto (Expo)	Alto	Muy alto

Cuadro 4.4: Comparativa de frameworks para el desarrollo cliente.

Justificación de la elección

Se ha seleccionado **React Native junto con Expo** frente a sus alternativas por las siguientes razones:

- **Experiencia Nativa:** Se descarta Ionic para evitar las limitaciones de rendimiento táctil y visual que imponen los *WebViews*, buscando ofrecer una experiencia de usuario genuinamente nativa.
- **Reutilización de paradigmas:** Al poseer experiencia previa en React, se elimina la curva de aprendizaje de un nuevo lenguaje como Dart. Esto permite invertir el tiempo del proyecto en resolver la lógica de negocio y optimizar las peticiones a la API.
- **Aceleración del ciclo de vida:** Las herramientas de empaquetado de Expo facilitan las pruebas continuas en dispositivos físicos mediante códigos QR, sin necesidad de lidiar con las configuraciones locales de Xcode o Android Studio durante el desarrollo.

IMPLEMENTACIÓN

En este capítulo veremos todo el proceso aplicado para implementar los diseños mostrados anteriormente. Veremos qué ha sido necesario hacer para poder construir todos los componentes que forman el sistema: el servidor back-end que hace de API, la BD, el Object Storage, Almacenamiento de Objetos (OS) y la interfaz de usuario.

5.1 Entorno de desarrollo

5.1.1 Editor de código

Para poder desarrollar el sistema se han utilizado dos editores de código distintos: uno para el front-end y otro para el back-end.

Para el front-end se ha decidido utilizar Visual Studio Code, ya que es uno de los editores más utilizados dentro de la industria. Además, este editor funciona de forma excelente para desarrollar aplicaciones basadas en entornos Java Script gracias a sus numerosas extensiones.

Mientras que para el back-end se ha optado por IntelliJ IDEA, un IDE de nicho cuando se trata de desarrollar sistemas implementados mediante tecnologías basadas en la JDK, como lo es en este caso Kotlin. Además, ofrece bastantes herramientas para trabajar de forma sencilla con lenguajes orientados a objetos, desarrollo de pruebas de código y ejecución y depuración de los programas desarrollados.

5.1.2 Ejecución y despliegue

El archivo *docker-compose.yaml* (ver en apéndice) se ha utilizado para configurar un entorno de desarrollo aislado y reproducible. Todo ello mediante el despliegue de contenedores para la comunicación con las distintas bases de persistencia de datos.

5.1.3 Migración

Para mantener una correcta gestión de migraciones para la base de datos relacional

5.2 Servidor back-end

Como se puede observar en la figura [...] se ha decidido aplicar una separación vertical por entidad. De esta forma la arquitectura del back-end se basa en un monolito modular. Como ya se ha explicado, se ha decidido aplicar una arquitectura de tres capas simple para los módulos de entidades con ciclos de vida simple; mientras que para módulos de entidades con ciclos de vida más complejos, se aplica arquitectura hexagonal.

5.3 Base de datos

Como ya hemos explicado, la BD ha sido implementada con el motor PostgreSQL. Se ha buscado cumplir la tercera forma normal para asegurar la mantenibilidad, unicidad e integridad de los datos en todo momento.

Para implementar el modelo de datos

La BD se ha implementado mediante los siguientes archivos .sql que dividen la creación de la BD en cuatro pasos:

1. **V1__init_schema:** crea todas las tablas necesarias para representar de forma correcta el modelo de datos.
2. **V2__add_soft_delete_triggers:** crea los disparadores necesarios para poder aplicar un sistema de borrado lógico sobre los recursos cuya eliminación física acarrearía estados no deseados dentro del sistema.
3. **V3__create_leaderboard_view:** crea la vista encargada de calcular las puntuaciones de los equipos para cada liga, grupo y jornada.
4. **V99__insert_initial_data:** inserta los datos necesarios para que el sistema pueda funcionar de forma correcta: los deportes disponibles, roles de usuario y una configuración y sistema de puntuación por defecto para ligas de voleibol.

Para la generación de los DTO en todos los servicios se han utilizado herramientas específicas cuya función principal es la generación de clases a partir de un contrato API. En el back-end, se ha utilizado la librería *org.openapi.generator*; y en el front-end, el paquete *swagger-typescript-api*.

5.4 Almacenamiento de objetos

5.5 Interfaz multiplataforma

CONCLUSIONES

6.1 Aprendizaje

Gracias a haber desarrollado este TFG se han aprendido y usado tecnologías que, aunque no se enseñe en el Grau d'Enginyeria Informàtica (GEIN), aportan conocimientos importantes sobre qué se utiliza hoy en día para crear aplicaciones mundialmente conocidas como lo es Instagram con React Native o Netflix con Spring Boot.

Principalmente, se ha aprendido a desarrollar un sistema cliente-servidor multi-plataforma aplicando los conocimientos adquiridos en el grado. Se ha aprendido una derivación de React (framework utilizado comúnmente para desarrollo web), para poder crear una interfaz capaz de utilizarse en cualquier dispositivo móvil y sus sistemas operativos correspondientes. Se ha aprendido un lenguaje de la misma familia de Java para poder evitar problemas sobre referencias nulas. Se ha utilizado un motor de base de datos ampliamente utilizado para poder adecuarse a las necesidades del mercado. Se ha aprendido a integrar un sistema de almacenamiento de objetos compatible con AWS, de manera local, para poder manejar tipos de datos no relacionales y mantenerlos con un alto nivel de privacidad. Y, por último, se ha aprendido a crear un entorno de desarrollo mediante Docker para tener un sistema consistente y fácilmente reproducible.

6.2 Mejoras aportadas con respecto a las alternativas

La principal mejora aportada es la implementación por defecto de las funcionalidades que las alternativas ofrecen después de hacer una inversión monetaria: imágenes de equipos y jugadores personalizadas, inclusión de roles árbitros y administradores ...

Otra aportación presente es la no necesidad de utilizar varias aplicaciones según el rol tal y como lo hace Clupik. Todo lo necesario para gestionar las ligas se puede hacer desde una misma aplicación. Incluso los capitanes no necesitan una aplicación

extra para acordar los partidos, ya que lo pueden hacer a través de la funcionalidad correspondiente que el propio sistema ofrece.

Una mejora es la posibilidad de autoalojar el sistema en un servidor propio, tan solo es necesaria la imagen de Docker que prepare el entorno de producción. Esto aporta privacidad y fiabilidad al no tener que depender de tecnologías o infraestructuras externas que pudieran fallar o cambiar sus condiciones de uso en cualquier momento. Sin embargo, siempre es posible migrar el sistema para alojarlo en la nube y poder delegar, automatizar y escalar tareas repetitivas como podrían ser el mantenimiento del sistema, la integridad de los datos a través de copias de seguridad o la autenticación mediante servicios externos.

6.3 Trabajo futuro

Si se quisiera continuar con el desarrollo de Sports League existen una gran cantidad de cosas que se podrían aplicar para mejorar la mantenibilidad y escalabilidad del sistema. En cualquier caso, nos centraremos en los aspectos críticos para conseguir un sistema que sea apto para ponerlo en producción dentro del ámbito de la UIB.

Lo primero de todo y más importante sería adquirir un certificado del Esquema Nacional de Seguridad (ENS): necesario para software enfocado a entidades públicas. Este tipo de certificación asegura que el sistema es confidencial, íntegro y disponible respecto a los datos, es seguro frente a amenazas y ciberataques y crea una confianza digital en el uso de medios públicos.

Para poder conseguir dicho certificado se deberían implementar tecnologías de trazabilidad y auditoría para poder tener un registro claro de todo lo que pasa dentro del sistema. Se integrarían tecnologías como SLF4J y Graphana para facilitar este tipo de tareas.

Otro punto importante sería aplicar la funcionalidad que no se ha podido aplicar por temas de seguridad: la verificación de credenciales de la UIB. Una vez que el sistema esté en pleno funcionamiento será posible integrar la API necesaria para dicha verificación mediante Open Authorization (OAuth). La parte positiva es que si esta funcionalidad opera correctamente sería posible implementar un sistema de registro e inicio de sesión directamente con las credenciales de la UIB.

Por otra parte, dado que el sistema no es de uso único para la UIB, la necesidad de credenciales de la UIB para participar en una liga sigue siendo una opción que se puede activar o desactivar según necesite el creador de la liga.



FORMULARIOS DE OPINIÓN

RESULTADOS DE LOS FORMULARIOS

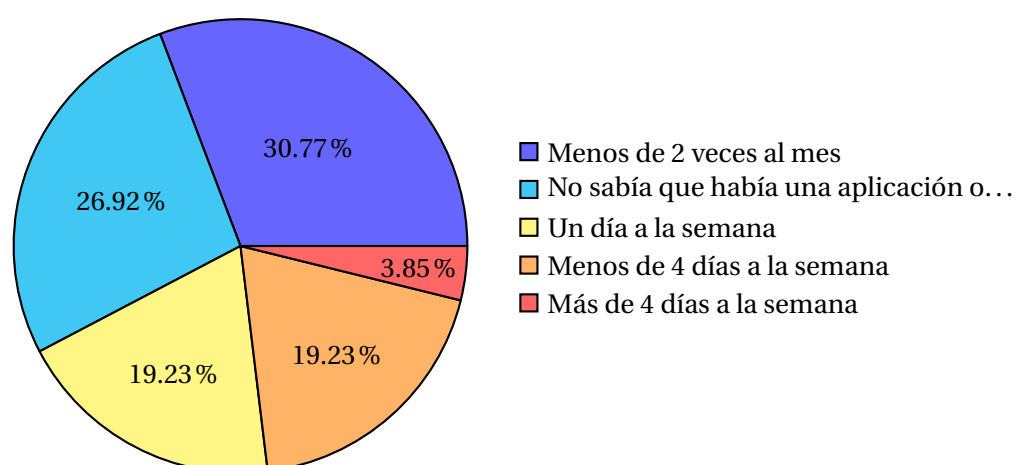


Figura B.1: ¿Cada cuánto usas la aplicación de la liga de la UIB o consultas la información de la liga a través de ella?

B. RESULTADOS DE LOS FORMULARIOS

¿Cómo puntuarías la administración de la liga a través de la aplicación?

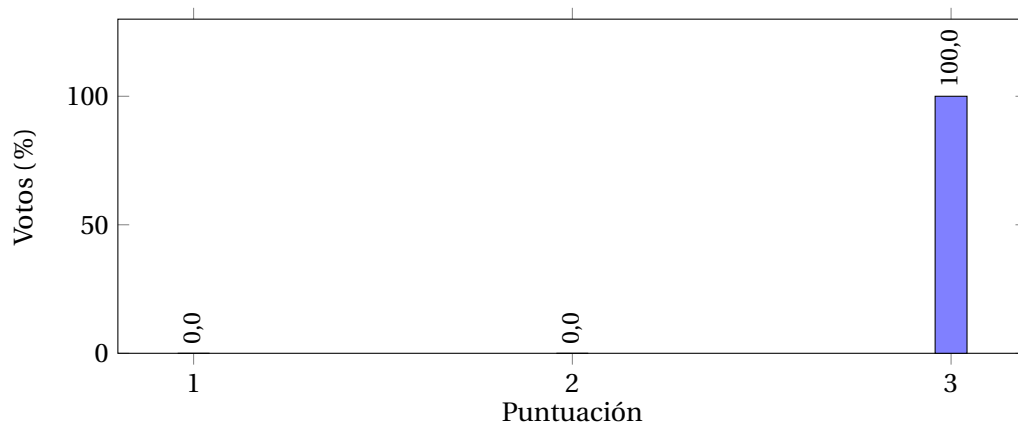


Figura B.2: ¿Cómo puntuarías la administración de la liga a través de la aplicación?

¿Cómo puntuarías la gestión de los partidos a través de la aplicación?

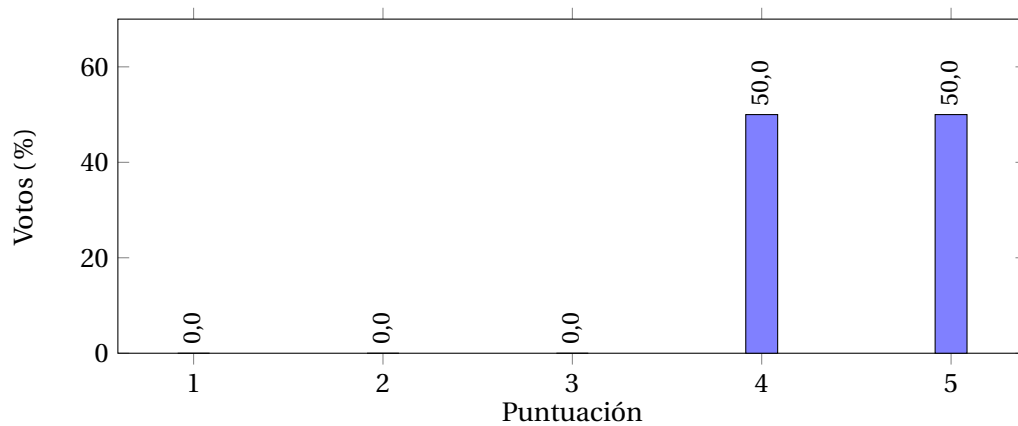


Figura B.3: ¿Cómo puntuarías la gestión de los partidos a través de la aplicación?

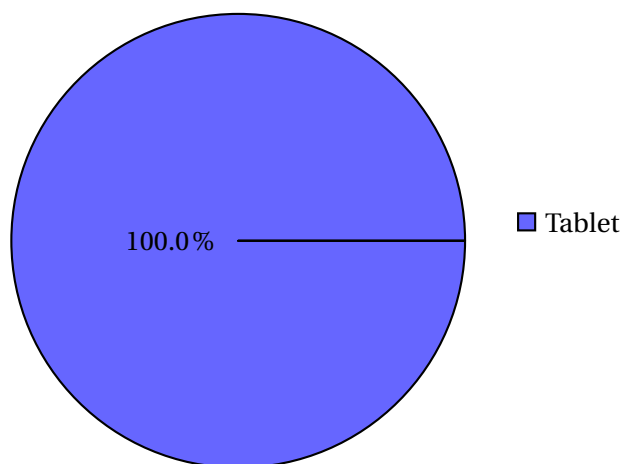


Figura B.4: ¿Usas algún dispositivo para rellenar el acta de los partidos?

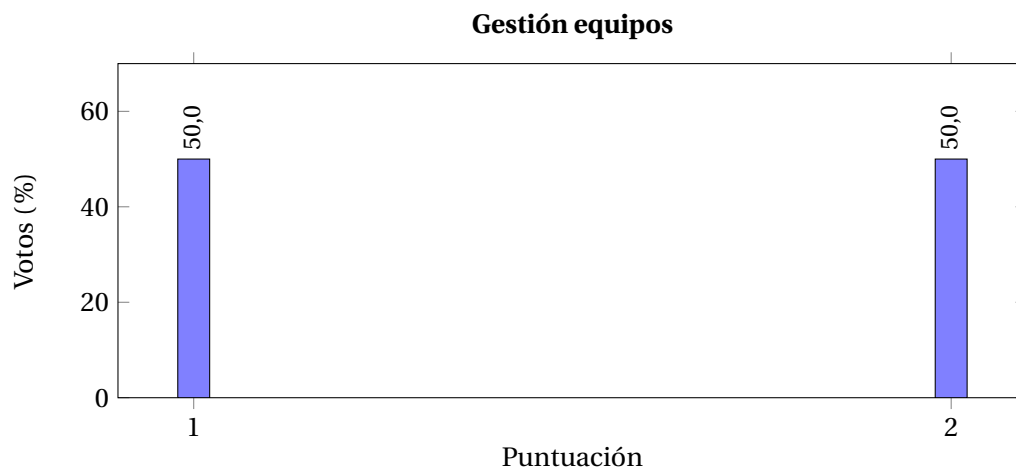


Figura B.5: Gestión equipos

Otros aspectos

- Me gustaría que hubiera estado más adaptada al voleibol. Se nota que no está hecha para este deporte por el tema de la complicación de asignar una distinta puntuación a diferentes finales de partido o la falta de aparición de los puntos a favor y él contra en la clasificación. Además la asignación de los árbitros a los partidos es muy tediosa por tener que seleccionarlos uno a uno. Me gustaría también que se creará un calendario donde se viera más claramente cuando es cada partido y árbitro asignado.
- Yo solo querría que cada equipo gestionase su plantilla y que los resultados y la clasificación se actualice.
- Que se puedan visualizar los puntos de cada equipo una vez finalizado el partido. Que tenga la opción de notificaciones para los próximos partidos.
- Quiero, que los puntos se repartan eficientemente, porque la aplicación está programada para una liga de fútbol y no para puntualizar marcadores de vóley.
- Quiero que se puedan personalizar los propios equipos porque me parece una buena forma de participación.

Problemas encontrados

- Me lío a veces al poner los resultados de los sets en cada apartado de los partidos.
- La aplicación que hay ahora solo se podía descargar en iPhone o era muy complicada de descargar.
- El desplegable de los grupos puede crear confusión, ya que no se modifica cuando has seleccionado otro deporte. Está predeterminado para el de fútbol. También en el apartado para ver los resultados deporte cada jornada (por fechas) solo está disponible los resultados de la primera jornada.

B. RESULTADOS DE LOS FORMULARIOS

- Hay funciones que no terminan de cargar como la bandeja de entrada o las notificaciones, va bastante lenta también.
- Me cuesta entrar porque no la veo nada práctica ni estética.
- El sistema de puntos suele ir mal, a veces diversos equipos han aparecido con otra cantidad de puntos que les corresponde a otros equipos.
- Hay errores en el tema de los partidos, por ejemplo hay partidos que no me salen de mi equipo o que aparecen como si no hubiesen pasado, pero ya han pasado varias semanas. No sé si es parte de la app o de quien mantiene la puntuación de cada partido, pero igual queda raro.



INTERFAZ WINNER

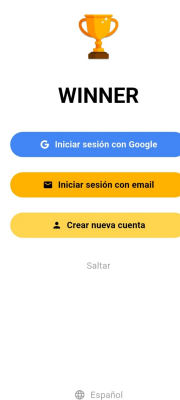


Figura C.1: Registro e inicio de sesión

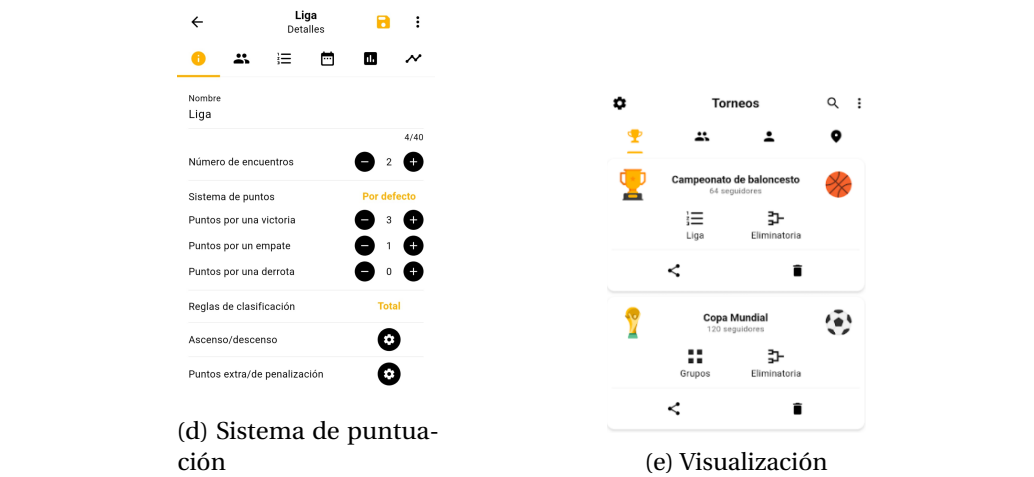
C. INTERFAZ WINNER



(a) Selección de deporte y formato de participación

(b) Deportes disponibles

(c) Personalización de fases



(d) Sistema de puntuación

(e) Visualización

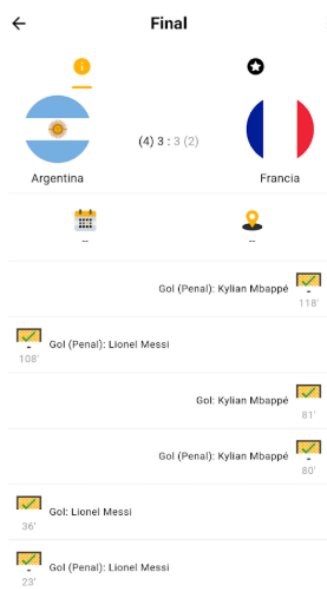


(a) Equipos

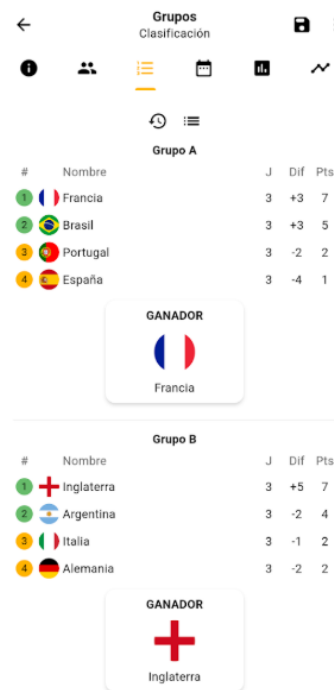
(b) Jugadores

(c) Personalización imagen

(d) Personalización color



(a) Resultado de partidos



(b) Clasificación



(a) Resultado de partidos



(b) Clasificación

Figura C.5



INTERFAZ CLUPIK

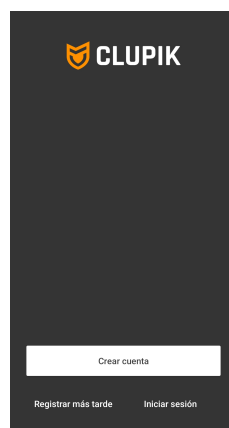
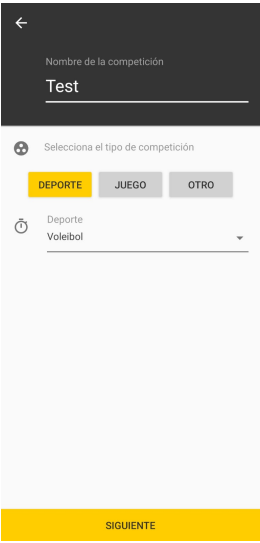
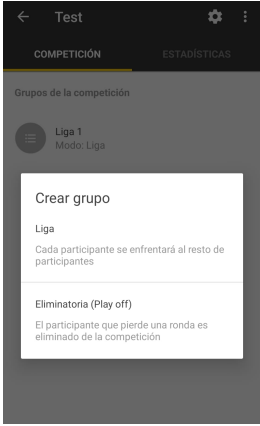


Figura D.1: Registro e inicio de sesión

D. INTERFAZ CLUPIK



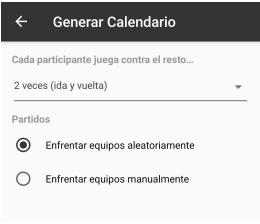
(a) Selección de deporte



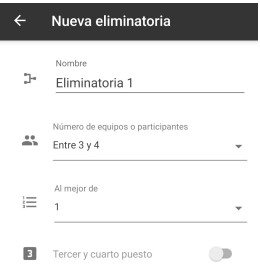
(b) Creación de fase



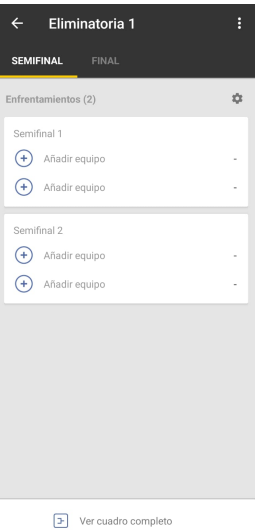
(c) Sistema de puntuación



(d) Fase de clasificación



(e) Fase de torneo



(f) Actualizar enfrentamientos

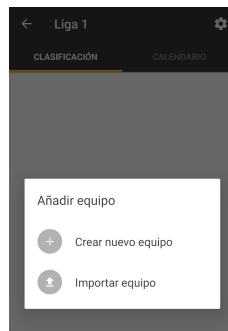
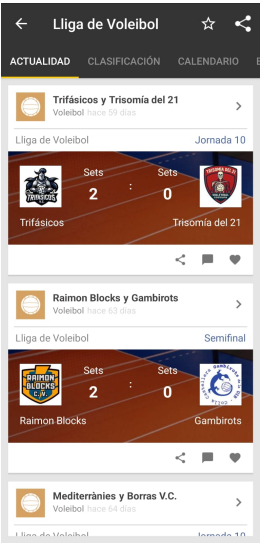
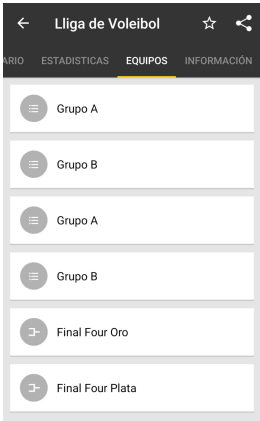


Figura D.3: Creación de equipos y jugadores

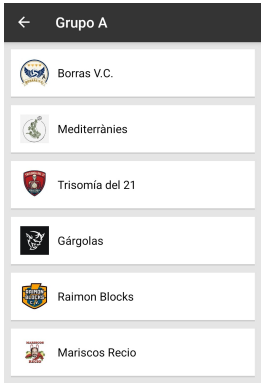
D. INTERFAZ CLUPIK



(a) Partidos disponibles



(b) Grupos



(c) Equipos



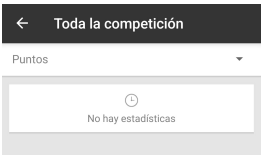
(d) Jornadas



(e) Partidos de la jornada

Grupo B												
Ps	Nombre	P	PJ	PG	PE	PP	F	C	D			
1	Red Birds	30	12	12	0	0	24	3	21			
2	Trifásicos	26	11	9	0	2	20	4	16			
3	Inter Erasmus	19	12	7	0	5	15	11	4			
4	Mopis VC	14	12	4	0	8	12	18	-6			
5	Super Campeones	12	10	5	0	5	10	13	-3			
6	Black Goat	10	12	3	0	9	7	18	-11			
7	Gambirots	1	11	0	0	11	1	22	-21			

(f) Clasificación por grupo



(g) Estadísticas

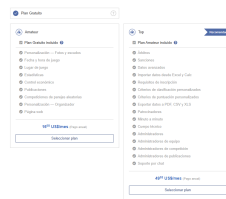
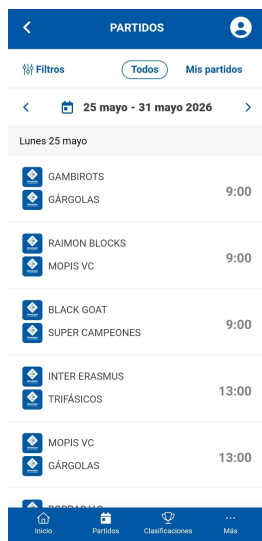
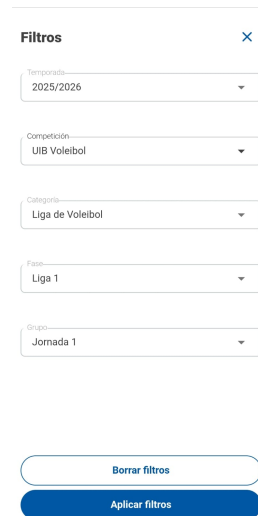


Figura D.5: Planes de pago



(a) Partidos disponibles



(b) Filtros

Figura D.6: Vista desde la aplicación de la UIB



DISEÑO DE LA BASE DE DATOS



CONTRATO API

BIBLIOGRAFÍA

- [1] Talent Apps, “Winner - Organizar torneos,” <https://play.google.com/store/apps/datasafety?id=il.talent.winner>, 2019, consultado: 2026-08-03. 1.1
- [2] Clupik, “Clupik - Software para clubes, academias y ligas,” Página web principal: <https://clupik.com/>, 2022, aplicaciones móviles disponibles: App para usuarios https://play.google.com/store/search?q=clupik&c=apps&androidId=111191616697929294962&utm_source=emea_Med. App para administradores https://play.google.com/store/apps/details?id=com.mileyenda.manager&utm_source=emea_Med. App para árbitros https://play.google.com/store/apps/details?id=com.mileyendareferee.app&utm_source=emea_Med. Consultado: 2026-08-03. 1.1
- [3] Robert C. Martin, “Design Principles and Design Patterns,” https://objectmentor.com/resources/articles/Principles_and_Patterns.pdf, 2009, consultado: 2026-08-25. 1.3